

Lean 4 at Qiuzhen College

Rongge Xu

January 2026

目录

0.1 前言	3
1 Getting started	4
1.1 安装 lean4	4
1.2 Overview	4
1.3 定理的结构	5
1.4 定理陈述中的参数	7
1.4.1 隐式参数与显式参数	7
1.4.2 补充: <code>def</code> 的陈述格式	8
1.4.3 方括号: 结构、 <code>class</code> 与类型类推断	8
2 Basic equations	10
2.1 基本计算和等式证明	10
2.1.1 <code>exact</code> 和 <code>rw</code>	10
2.1.2 <code>apply</code> 和 <code>rw</code>	11
2.1.3 <code>section</code> 与 <code>variable</code>	13
2.2 exercises	15
3 化简	18
3.1 交换环上的化简 (<code>ring</code>)	18
3.2 不等式与格论	19
3.2.1 格论 (Lattice): 统一的序结构接口	20
4 结构	23
4.1 逻辑结构与证明策略	23
4.1.1 第一类: 当逻辑符号在“目标”中	23
4.1.2 第二类: 当逻辑符号在“假设”中	24
4.1.3 逻辑变换与自动化	25
4.1.4 总结: 逻辑操作速查表	26
4.1.5 自动化简化: <code>simp</code>	26
4.1.6 结构化证明: <code>calc</code> 模式	27
5 Mathlib 指南	30
5.1 寻找 Mathlib 定理	30
5.2 Lean 中的命名法	31
5.2.1 常见符号与缩写速查表	32

5.2.2	命名空间 (Namespaces)	34
5.3	Mathlib 代码风格指南	35
5.3.1	排版与空白 (Layout & Formatting)	35
5.3.2	显式与隐式参数 (Explicit vs Implicit Arguments)	35
5.3.3	证明风格 (Proof Style)	36
5.3.4	文档规范 (Documentation)	37
6	类型的层级	38
6.1	Universe	38
6.1.1	类型的类型	38
6.1.2	宇宙层级	38
6.2	核心机制：函数类型	39
6.2.1	一元函数与 λ 构造	39
6.2.2	多元函数的本质：柯里化 (Currying)	39
6.2.3	依赖函数类型 (Π -type)	40
6.3	逻辑视角：命题即类型	40
6.3.1	蕴含即函数 ($\rightarrow$)	40
6.4	证明：Term vs Tactic	41
6.4.1	Term Proof：直接构造	41
6.4.2	Tactic Proof：指令式构造	41
6.4.3	show_term：揭示真相	41
6.5	Exercises	43
6.5.1	命名	43
6.5.2	tactics	43
7	Dot Notation	46
7.1	命名空间与点方法	46
7.1.1	命名空间 (Namespaces)	46
7.1.2	点方法 (Field Notation)	48
8	逻辑表达	50
8.1	Lean 中的逻辑表达	50
8.1.1	命题逻辑	50
8.2	一些进阶 Tactics	54
8.2.1	suffices, assumption 与 refine	54
8.2.2	函数相关：funext, apply_fun 与 gcongr	56
8.2.3	逻辑命题相关的实用 Tactic	58
9	归纳与递归	61
9.0.1	归纳类型	61
9.0.2	模式匹配	64
10	集合与子类型	67
10.1	集合	67
10.1.1	集合的定义	67
10.2	子类型 (Subtype) 与类型转换	74

10.2.1 子类型的定义与结构	74
10.2.2 类型转换 (Coercion)	75
10.2.3 类型转换处理 tactic	76
10.3 更多自动化 Tactic	78
10.3.1 外延性 (ext)	78
10.3.2 数值判定与代数化简	78
10.3.3 综合性求解器	79
11 群	82
11.1 Mathlib 中的群结构	82
11.2 群同态、子群与商群	83
11.2.1 群同态 (Group Homomorphisms)	83
11.2.2 子群 (Subgroups)	85
11.2.3 正规子群与商群 (Quotient Groups)	87
11.2.4 构造积群同态	88
11.3 Exercises	89

0.1 前言

此课件面向参加 2026 年 1 月清华大学求真书院一周 Lean4 workshop 的同学，假设读者具备一定数学基础。我们的目标是用尽量短的时间帮助大家掌握 Lean 语言与交互式证明的核心工作流（基本语法、常用 tactic、以及与数学对象相关的表达方式），从而在 workshop 结束后能够借助 AI 工具，开始参与有限单群分类（CSFG, Classification of Finite Simple Groups）相关材料的形式化与标注工作。

本讲义主要参考 *Mathematics in Lean* 以及姜杰东老师于 2025 年 1 月在西湖大学组织 Lean4 workshop 的未发表讲义（在此特别致谢）。

与本次课程配套的报告、作业与代码仓库见：

https://github.com/ronggexu/lean4_workshop_26winter

Chapter 1

Getting started

1.1 安装 lean4

参考 https://github.com/ronggexu/lean4_workshop_26winter/blob/main/LEAN_INSTALL.md 安装 lean4:v4.26.0

```
#eval "Hello, World!"
```

本教程使用 Lean 4 及其主流数学库 **mathlib**。我们默认在文件开头，

```
import Mathlib
```

它会导入 mathlib 中已经形式化好的大量定义、定理与证明工具。

常用链接

- Lean community 主页: <https://leanprover-community.github.io>
- Lean 社群 Zulip 论坛: <https://leanprover.zulipchat.com>
- Lean Game Server: <https://adam.math.hhu.de>
- 在线使用 Lean: <https://live.lean-lang.org/>
- Mathlib 文档: <https://leanprover-community.github.io/mathlib4-docs>

1.2 Overview

我们先建立一个最重要的直觉：

在 Lean 中，任何表达式都有唯一的类型。

例如，数字有类型。可以看到。默认情况下，Lean 会把 1 视为自然数， $\sqrt{2}$ 的类型则是实数 $\mathbb{R}$ 。

```
#check 1      -- 1 : ℕ
#check √2     -- √2 : ℝ
```

你可以用 `#check` 命令查看某个表达式的类型：把表达式写在 `#check` 后面，然后在 `infoview` 中读取输出。Lean 的输出通常是 “A : B” 的形式：冒号左边是项 (term)，右边是它的类型 (type)。

```
-- Every expression has a type, you can use the #check command to print it.
#check 1 + 1

def f (x : ℕ) :=
  x + 3

#check f
```

例如上面 $1 + 1$ 是一个表达式，因此也有类型；而 f 是一个函数定义（输入 $x : \mathbb{N}$ ，输出 $x + 3$ ），因此 `#check f` 会显示它的函数类型。

除了“数据”和“函数”，**命题**在 Lean 中也同样是表达式，并且它们的类型是 `Prop`：

```
-- These are propositions, of type 'Prop'.
#check 2 + 2 = 4

def myFermatLastTheorem :=
  ∀ x y z n : ℕ, n > 2 ∧ x * y * z ≠ 0 → x ^ n + y ^ n ≠ z ^ n

#check FermatLastTheorem
```

这里 $2 + 2 = 4$ 是一个命题，因此它的类型是 `Prop`。同理，`FermatLastTheorem` 虽然看起来像“一个很长的句子”，但它仍然只是一个表达式：它的值是一个命题，所以它的类型也是 `Prop`。

更进一步，Lean 把“证明”也当作表达式：如果 P 是一个命题（即 $P : \text{Prop}$ ），那么“证明 P ”就是一个类型为 P 的项。换句话说，证明的类型就是它所证明的命题：

```
-- Some expressions have a type, P, where P itself has type Prop.
-- These are proofs of propositions.
theorem easy : 2 + 2 = 4 :=
  rfl

#check easy

theorem hard : FermatLastTheorem :=
  sorry

#check hard
```

例如，`easy` 是一个项，它的类型是命题 $2 + 2 = 4$ ；这表示 `easy` 就是该命题的一个证明。这里的 `rfl` 可以理解为“反身性证明”：当等式两边在化简后相同，`rfl` 往往就能直接完成证明。类似地，`hard` 的目标类型是 `FermatLastTheorem`，此处先用 `sorry` 占位（表示“暂时承认”）；在正式成果或作业提交中，一般应避免保留 `sorry`。

从这些例子可以看到：Lean 里“数学对象 / 命题 / 证明”都统一成“项 + 类型”的语言。接下来我们会逐步解释：常见的数学概念在 Lean 的类型论框架下分别如何表达与使用。

1.3 定理的结构

下面我们看一个“完整的 Lean 定理 + 证明”的例子：

```
theorem The_first_example {G : Type*} [Group G] {a b c d e f : G} (h : a * b = c * d) (h' : e = f) : a * (b * e) = c *
  (d * f) := by
```

```
rw [h']
rw [← mul_assoc]
rw [h]
exact mul_assoc c d f
```

建议你先在本机运行这段代码，并在编辑器中点选不同位置观察 `infoview`：你会看到当前有哪些参数与假设，以及当前需要证明的目标如何随证明步骤变化。

从整体结构上看，Lean 中一个定理由两部分组成：

- **陈述 (statement)**：说明在什么参数与假设下，要证明什么结论；
- **证明 (proof)**：给出完成该结论的证明脚本。

本小节先聚焦在**陈述部分**；证明脚本的细节（例如 `rw`、`exact` 等 `tactic`）会在后续章节讲解。

读懂定理陈述的骨架

把上面第一行按结构拆开，你会发现它基本就是下面的模板：

```
theorem 名称 参数/假设 : 目标 := by 证明
```

逐段解释：

- **theorem**：声明接下来要给出一个定理（并提供其证明）。
- **The_first_example**：定理名字，由你自己命名（同一文件/命名空间内不能重名）。
- **{G : Type*} [Group G] {a b c d e f : G} (h : ...) (h' : ...)**：参数与假设。
- **: a * (b * e) = c * (d * f)**：目标（要证明的结论）。注意：定理的类型就是它的结论。
- **:= by**：从这里开始进入证明部分。可以暂时把它当作一个固定搭配：`:=` 表示“接下来给出这个定理（这个项）”，`by` 表示“用 `tactic` 风格写证明”。

就数学含义而言，这个定理在说：“设 G 是一个群， $a, b, c, d, e, f \in G$ 。给定假设 $h : ab = cd$ 与 $h' : e = f$ ，则有 $a(be) = c(df)$ 。”

infoview 在证明过程中显示什么

在证明过程中，`infoview` 会持续展示“当前上下文”和“当前目标”。例如当光标停在 `rw [h']` 之后，`infoview` 可能显示：

```
/-
G : Type u_1
inst1 : Group G
a b c d e f : G
h : a * b = c * d
h' : e = f
⊢ a * (b * f) = c * (d * f)
-/
```

符号 $\vdash$ 右边是当前需要证明的目标，左边（上面）是当前可用的参数与假设。随着证明步骤推进，目标会逐步被改写直至出现 `No goals`，表示证明完成。

同一例子：把陈述部分标出来

下面把同一个例子在第一行用注释标出陈述的几个区块(注意:为避免重名,这里把定理名改成 `The_first_example'`):

```
theorem /-name-/The_first_example' /-parameters-/ {G : Type*} [Group G] (a b c d e f : G) (h : a * b = c * d) (h' : e =
  f) : /-goal-/ a * (b * e) = c * (d * f) := by /-proof-/
sorry
```

小结

一个定理 = 陈述 + 证明。其中陈述部分的基本写法是:

theorem + 名称 + 参数/假设 + : + 目标

理解并熟悉这一骨架后,我们就可以逐步学习:如何写参数(花括号/方括号/圆括号的区别)、如何组织假设,以及如何用 tactic 构造证明。

1.4 定理陈述中的参数

本节解释定理陈述中三种括号的含义:花括号 `{ }`、圆括号 `( )`、方括号 `[ ]`。核心结论先说在前面:

- `{...}`: 隐式参数 (通常可由 Lean 自动推断,使用时一般不需要手动填写);
- `(...)`: 显式参数 (使用时通常需要手动提供);
- `[...]`: 类型类参数 (要求某个类型携带特定结构,Lean 会用类型类推断自动寻找实例)。

1.4.1 隐式参数与显式参数

先看一个例子:

```
theorem my_lt_trans {a b c : ℕ} (h1 : a < b) (h2 : b < c) : a < c := by
  apply lt_trans
  apply h1
  apply h2
```

这里同时出现了花括号和圆括号:它们都在给定理提供“输入”,区别在于

- `{a b c : ℕ}` 是隐式参数:Lean 通常可以从上下文推断出它们;
- `(h1 : a < b) (h2 : b < c)` 是显式参数:它们是我们明确给定的假设,作为定理的输入。

理解方式可以把“定理”看作一个把若干输入映射到结论的对象:给定所需的假设,就能得到结论。比如 `lt_trans` 的直觉就是“如果 $a < b$ 且 $b < c$, 则 $a < c$ ”。在 Lean 里,你可以直接把两个证明喂给它:

```
theorem one_lt_two' : 1 < 2 := by sorry
theorem two_lt_three' : 2 < 3 := by sorry

example : 1 < 3 := lt_trans one_lt_two' two_lt_three'
```

注意：这里我们没有手动提供 a, b, c 。原因是，当你给出 `one_lt_two' : 1 < 2` 与 `two_lt_three' : 2 < 3` 时，其中已经包含了 $a = 1, b = 2, c = 3$ 这些信息，Lean 可以自动从这两个假设中把隐式参数推断出来。因此在陈述 `my_lt_trans` 时，我们把 a, b, c 放进花括号里，表示：这些参数在使用定理时通常不需要显式填写，让 Lean 自动推断即可。

顺便一提，上面出现的 `example` 和 `theorem` 类似，区别在于 `example` 不需要命名；我们会在后续再细讲它们的用途差异。

1.4.2 补充：def 的陈述格式

除了定理（`theorem`），我们也经常需要定义函数/对象（`def`）。`def` 的骨架与 `theorem` 很像，同样是“关键词 + 名称 + 参数 + : + 类型 + := + 内容”。区别在于：

- `theorem` 的“: 后面”是要证明的命题，而“:= by 后面”是证明；
- `def` 的“: 后面”是返回值的类型，而“:= 后面”是具体返回的对象/表达式。

例如：

```
def div_two {a : ℤ} (h : Even a) : ℤ := a / 2
```

这里 `{a : ℤ}` 是隐式参数，`(h : Even a)` 是显式参数；整体表示：给定一个整数 a （通常可从上下文推断）以及它是偶数的证明 h ，返回整数 $a / 2$ 。

1.4.3 方括号：结构、class 与类型类推断

现在解释方括号 `[ ]`。它通常出现在形如 `[Group G]`、`[TopologicalSpace X]` 这样的地方，表示：我们要求某个类型携带一份由 `class` 声明的结构，并让 Lean 自动寻找对应的实例（instance）。

为了理解这个机制，需要先区分 `structure` 与 `class`：

- `structure` 用来把若干数据/性质“打包”成一个新的结构（你需要显式构造它）；
- `class` 也是打包结构，但它会进入 Lean 的“类型类推断系统”：当你写 `[SomeClass α]` 时，Lean 会尝试自动合成（synthesize）出该结构在 α 上的实例。

用 `structure` 打包信息 例如三维点可以用三个坐标打包：

```
structure Point where
  x : ℝ
  y : ℝ
  z : ℝ
```

再例如“线性”可以理解为一个函数满足若干性质（这里用两个性质做演示）：

```
structure IsLinear (f : ℝ → ℝ) where
  is_additive : ∀ x y, f (x + y) = f x + f y
  preserves_mul : ∀ x c, f (c * x) = c * f x
```

用 `class + instance` 让 **Lean** 自动“记住”结构 当某些结构在特定类型上是约定俗成的（例如 $\mathbb{N}$ 上的 $\leq$ ），我们通常希望 **Lean** 自动使用默认实例，而不是每次都手动传入。这就是 `class` 的用途：把结构交给类型类系统管理，并用 `instance` 为具体类型登记实例。

```
-- Remember that a partial order is a binary relation that is transitive, reflexive, and antisymmetric. An even weaker
  notion sometimes arises: a preorder is just a reflexive, transitive relation.

class PreOrder' ( $\alpha$  : Type*) where
  le :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$  -- 它接收两个类型为  $\alpha$  的元素，并返回一个命题
  le_refl :  $\forall a : \alpha, \text{le } a a$ 
  le_trans :  $\forall a b c : \alpha, \text{le } a b \rightarrow \text{le } b c \rightarrow \text{le } a c$ 

instance : PreOrder'  $\mathbb{N}$  where
  le := sorry
  le_refl := sorry
  le_trans := sorry

#synth PreOrder'  $\mathbb{N}$ 
```

上面这段的阅读方式是：

- `class PreOrder'` 定义了“预序”所需的数据与公理；
- `instance : PreOrder'  $\mathbb{N}$` 登记了自然数上的一个预序实例（这里用 `sorry` 占位）；
- `#synth PreOrder'  $\mathbb{N}$` 让 **Lean** 尝试自动合成该实例，用来检验登记是否成功。

回到 `[Group G]` 的含义 因此，当你在定理陈述里写 `[Group G]` 时，含义就是：假设 G 携带一个群结构（这是一个由 `class` 声明的结构），并让 **Lean** 在需要时自动从上下文中找到它。这正是第一节例子里 `{G : Type*} [Group G]` 的意义。

小结

- 花括号 `{...}`：隐式参数，**Lean** 通常可推断，使用时一般不需手动填写；
- 圆括号 `(...)`：显式参数，使用时通常需要你明确提供；
- 方括号 `[...]`：类型类参数，要求某个 `class` 结构，并由类型类推断自动寻找 `instance`。

此外，`def` 的陈述骨架与 `theorem` 类似，都是“关键词 + 名称 + 参数 + : + 类型 + := + 内容”，只是 `theorem` 的内容是证明，而 `def` 的内容是返回值本身。

Chapter 2

Basic equations

2.1 基本计算和等式证明

要在 Lean 中证明定理，你需要先掌握两件事：

- 可用的已有定理来自哪里：大量基础定义与常用引理都已收录在 **mathlib** 中，并按数学分支分文件组织。
- 如何把这些定理用起来：Lean 提供了一套面向证明过程的脚本语法，称为 **tactic**。除了 tactic 风格，Lean 也支持直接写“项”的证明（term proof）；我们后面会比较两者的差别与各自适用场景。

本节先介绍五个最常用、足以覆盖大量入门证明的 tactic：**exact**、**rfl**、**apply**、**rw** 和 **have**。掌握它们之后，你就可以开始独立形式化许多简单的计算与等式类定理。

2.1.1 exact 和 rfl

exact

当我们的目标已经是库中某个定理（或引理）的结论时，可以用 **exact** 直接把它“交上去”，从而一步完成证明。例子如下：

```
#check mul_assoc
example (a b c : R) : a * b * c = a * (b * c) := by
  exact mul_assoc a b c
```

这里 `mul_assoc` 是乘法结合律。通过 `#check mul_assoc` 你会看到它需要三个显式参数，并给出结论 $a * b * c = a * (b * c)$ ¹。在上面的 `example` 中，我们的目标恰好就是这个结论（只是把变量具体命名为 `a b c`），因此把参数填好以后，`mul_assoc a b c` 的类型正是我们要证明的目标，于是 **exact** 可以一步结束证明。

需要强调的是：**exact** 只能在你给出的项的类型与当前目标完全一致时使用。也就是说，它本质上是“一步收尾”的 tactic：你必须已经拿到了一个能直接证明当前目标的项（通常就是某个定理应用到合适参数后的结果）。如果你希望“先用某个定理把目标化简/转化为另一个子目标”，但还不能立刻结束证明，那么就应当使用 **apply**（**exact** 可以看作 **apply** 的特殊情况），我们马上会介绍它。

¹不加括号的时候默认函数应用（Application）左结合（eg: $a * b * c := (a * b) * c$ ），函数类型（Function Types）右结合（eg: $\mathbb{R} \rightarrow (\mathbb{R} \rightarrow \mathbb{R})$ ）

rfl

另一个最常见的“收尾” tactic 是 rfl。它用于证明形如“左右两边本来就是同一个表达式”的等式，例如：

```
example (x y : ℝ) : x + 37 * y = x + 37 * y := by rfl
```

表面上看这有些“平凡”，但 rfl 的价值远不止于此：在许多情况下，等式两边经过定义展开或化简后会变得完全相同，此时 rfl 也能直接完成证明。我们后面会用更有代表性的例子展示这一点。

2.1.2 apply 和 rw

apply

apply 与 exact 都是“使用已有定理”的 tactic。最简单的情形是：你把定理连同所需参数都填好，使其结论恰好与当前目标一致，那么 apply 也能一步结束证明：

```
theorem my_lt_trans' {a b c : ℕ} (h1 : a < b) (h2 : b < c) : a < c := by
  apply lt_trans h1 h2
```

回忆 lt_trans 表达“ $(a < b)$ 且 $(b < c)$ 推出 $(a < c)$ ”。当我们把 h1 和 h2 依次喂给它时，就得到了目标 $a < c$ ，因此证明完成。直觉上，apply 做的是一种**反向推理**：它先匹配引理的结论与当前目标，然后把引理的前提变成需要继续证明的子目标（如果这些前提不能立刻由你提供的话）。

与 exact 的关键差别在于：apply 更灵活——你不必一次把所有参数都填满。Lean 会尝试自动匹配/推断；推不出来的部分，会被转化为新的目标，留待后续继续证明。这就使得 apply 适合“先把大目标拆成更小的目标”。

参数填写与占位符

为了准确理解 apply 的行为，这里补充几条规则：

- **参数按顺序提供**。使用一个定理时，参数的填写顺序与该定理陈述中的参数顺序一致。
- **显式参数可以暂时不填**。在 apply 中，如果某个显式参数 Lean 无法自动推断，它会变成新的证明目标。
- **隐式参数通常不需要（也不建议）直接按位置填写**。如果你确实要指定某个隐式参数，可以用“命名赋值”的方式，例如 $(b := b)$ 。
- 如果你想“跳过某个位置稍后再填”，可以用**占位符**：
 - `_`：让 Lean 尝试自动推断该位置；
 - `?_`：在该位置生成一个新的目标，稍后再证明/填入。

下面的例子分别展示：指定隐式参数、不填显式参数让参数变成子目标、以及用 `?_` 跳过参数创建子目标。

```
#check lt_trans

theorem my_lt_trans1 {a b c : ℕ} (h1 : a < b) (h2 : b < c) : a < c := by
  apply lt_trans (b := b) -- 指定（原本可推断的）参数（此处第一个b是lt_trans中的命名）
  apply h1
  apply h2

-- 不显式提供参数时，Lean 会尝试推断；推不出来的部分会变成新的目标
theorem my_lt_trans2 {a b c : ℕ} (h1 : a < b) (h2 : b < c) : a < c := by
```

```

apply lt_trans
apply h1
apply h2

-- 需要按顺序填参；用 ?_ 跳过的位置会生成新目标
theorem my_lt_trans3 {a b c : ℕ} (h1 : a < b) (h2 : b < c) : a < c := by
  apply lt_trans ?_ h2
  apply h1

```

在实际写证明时，你可以把 `apply` 的使用理解为：“我想用这个引理把当前目标变成它的前提；前提如果我已经有了，就直接提供；如果没有，就让 Lean 把它变成新的目标，我再继续证明。”

`rw`

下面介绍 `rw`。它是 *rewrite* 的缩写，用来按等式（或等价）改写当前目标。先看最基本的例子：

```

example {a b c d : ℝ} (h1 : a = c) (h2 : b = d) : a * b = c * d := by
  rw [h1]
  rw [h2]

```

`rw [h1]` 的直觉是：在当前目标中寻找与等式 $h1 : a = c$ 左边 a 相同的部分，并用右边 c 替换它。因此第一步会把目标 $a * b = c * d$ 改写成 $c * b = c * d$ ；第二步再用 $h2 : b = d$ 把 b 改写成 d ，从而完成证明。

方括号 [...] 在 `rw` 里的含义 这里的 `[ ... ]` 表示“改写规则的列表”（可以写一个或多个，用逗号分隔），例如 `rw [h1, h2]` 等价于连续写两行 `rw [h1]`、`rw [h2]`。注意这与类型声明里的 `[Group G]` 不同：`[Group G]` 是类型类参数，而 `rw [ ... ]` 只是 tactic 的参数列表。

改写方向：`rw [h]` 与 `rw [←h]` 默认情况下 `rw [h]` 是“把等式左边替换成右边”。如果你希望反过来用（用右边替换成左边），可以写 `rw [← h]`：

```

#check mul_add
--#check (mul_add a b c : a * (b + c) = a * b + a * c)
example (a : ℝ) : a * 0 + a * 0 = a * 0 := by
  rw [← mul_add, add_zero]

```

这里 `mul_add` 的结论形如 $a * (b + c) = a * b + a * c$ 。我们的目标左边出现的是 $a * 0 + a * 0$ ，它更像是该等式的右边 ($a * b + a * c$) 的形状，而我们希望把它改写成左边 ($a * (b + c)$) 的形状，于是使用 `← mul_add` 反向改写。随后 `add_zero` 把 $0 + 0$ 化简为 0 ，等式两边就一致了。

不给参数 vs 指定参数：改写发生在哪里？`rw` 里使用的定理（或假设）通常也带参数。你可以不显式提供参数，让 Lean 自动匹配；也可以给出参数，从而“指向”你希望改写的位置。

```

#check mul_comm
example (a b c : ℝ) : a * (b * c) = b * c * a := by
  rw [mul_comm]

example (a b c : ℝ) : a * (b * c) = a * (c * b) := by
  rw [mul_comm b c]

```

解释如下：

- 第一个例子里写 `rw [mul_comm]`, Lean 会在目标中寻找**第一个**能匹配乘法交换律的乘法子表达式 (这里是 $a * _$), 因此它会在最外层把 $a * (b * c)$ 改写成 $(b * c) * a$, 也就是右边的 $b * c * a$ 。
- 第二个例子写 `rw [mul_comm b c]`, 相当于明确告诉 Lean 你要用的是 $b * c = c * b$ 这条等式, 于是它会在 $(b * c)$ 这个位置进行改写, 把 $a * (b * c)$ 改写成 $a * (c * b)$ 。

总之, `rw` 的使用可以概括为: 用一个等式 (或等价) 在当前目标中做替换; 默认左到右, 必要时可用 $\leftarrow$ 反向; 不给参数则自动找位置, 指定参数则更精确地控制改写位置。

2.1.3 section 与 variable

`variable` 用来把一些将反复出现的变量提前声明出来, 避免在每个定理陈述里重复书写。它常与 `section ... end` 搭配使用: `section` 与 `end` 划定了一个局部作用域——在该区域内声明的 `variable` 只对本段落中的定义与定理有效, 不会影响 `end` 之后的内容。

```
section have

variable (a b c d e f : R)

#check add_mul
example (h : a + b = c) : (a + b) * (a + b) = a * c + b * c := by
  nth_rw 2 [h]
  rw [add_mul]

#check add_left_cancel
example : a * 0 = 0 := by
  have h : a * 0 + a * 0 = a * 0 + 0 := by
    rw [← mul_add, add_zero, add_zero]
  rw [add_left_cancel h]

end have
```

这段代码里出现了两个新 tactics:

`nth_rw`: 只改写第 n 处匹配

第一个例子里用到了 `nth_rw`, 它可以看作 `rw` 的变体: 当目标中有多处都能匹配某个等式时, `nth_rw n [h]` 只改写其中第 n 处匹配的位置。

在该例中, 目标里出现了两次 $(a + b)$, 我们只想把第二个改写成 c , 于是写:

- `nth_rw 2 [h]`: 把目标中第 2 处与 $a + b$ 匹配的子表达式, 用 c 替换;
- `rw [add_mul]`: 再用分配律把 $(a + b) * c$ 展开为 $a * c + b * c$ 。

`have`: 先证一个引理, 再回到原目标

第二个例子展示了 `have` 的典型用法。我们要证明 $a * 0 = 0$, 一种常见策略是先把它转化为一个可用“加法消去”的形式: 构造一个等式 $a * 0 + a * 0 = a * 0 + 0$, 然后用 `add_left_cancel` 消去左边公共项得到结论。

`have` 的格式是

`have 名称: 断言 := by 证明`

它会暂时把当前目标切换为该断言：当你完成这段 `by` 证明后，`have` 产生的引理会以给定名称加入上下文，随后你回到原目标继续证明。

在例子中：

- `have h : ... := by ...` 先证明中间引理 `h`；
- 证明完成后，`h` 出现在上下文里；
- `rw [add_left_cancel h]` 使用 `add_left_cancel` 把等式两侧的 $a * 0$ 消去，从而得到 $a * 0 = 0$ 。

如果你省略 `have` 的名称（不写 `h`），Lean 会把它默认命名为 `this`，你随后可以用 `this` 引用该引理。

2.2 exercises

natural number game

<https://adam.math.hhu.de/#/g/leanprover-community/nng4>

check command

看看如下`#check` 命令在 `infoview` 分别输出什么并理解原因:

```
section
variable (a b c : ℝ)

#check (a : ℝ)
#check (a : ℕ)
#check (mul_comm a b : a * b = b * a)
#check mul_assoc c a b
#check mul_comm a
#check mul_comm

end
```

按定义相等

```
example (a b : ℝ) : a - b = a + -b := by
  sorry
```

Using rw

只用 `rw tactic` 和 `#check` 给出的定理完成如下习题

```
section
variable (a b c d f e: ℝ)

#check lt_trans
#check mul_assoc
#check mul_comm

example : c * b * a = b * (a * c) := by
  sorry

example : a * (b * c) = b * (a * c) := by
  sorry

example (h : b * c = e * f) : a * b * c * d = a * e * f * d := by
  sorry
```

```

#check sub_self

example (hyp : c = b * a - d) (hyp' : d = a * b) : c = 0 := by
  sorry

#check mul_add
#check add_mul
#check add_assoc
#check two_mul

example : (a + b) * (a + b) = a * a + 2 * (a * b) + b * b := by
  sorry

#check add_zero
#check pow_two
#check mul_sub
#check add_sub
#check sub_sub

example : (a + b) * (a - b) = a ^ 2 - b ^ 2 := by
  sorry

end

```

rw 结合其他

使用rw, 考虑结合 exact, nth_rw, have, apply完成如下练习

```

example (a b c d : ℝ) (hyp : c = d * a + b) (hyp' : b = a * d) : c = 2 * a * d := by
  sorry

example (a b c : ℕ) (h : a + b = c) : (a + b) * (a + b) = a * c + b * c := by
  sorry

#check add_left_cancel
example (a : ℝ) : 0 * a = 0 := by
  sorry

#check neg_add_cancel_left

theorem neg_eq_of_add_eq_zero {a b : ℝ} (h : a + b = 0) : -a = b := by
  sorry

theorem neg_neg' (a : ℝ) : - -a = a := by
  sorry

#check inv_mul_cancel
#check one_mul

```

```
theorem my_mul_inv_cancel {G : Type*} [Group G] (a : G) : a * a-1 = 1 := by
  sorry
```

我们会在 Day 2 引入更多和 `apply` 相关的练习.

Chapter 3

化简

3.1 交换环上的化简 (ring)

上一节我们用 `rw` 逐步改写等式来做计算,但很多时候我们其实不必手动一条条 `rw`。比如对于交换环(如 $\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$) 中的大量“纯代数等式”, `mathlib` 提供了一个自动化 tactic: `ring`。

环结构与可用的基本事实 在 Lean 中, `[Ring R]` 表示类型 `R` 带有环结构, 它提供:

```
variable (R : Type*) [Ring R]
-- 加法
#check (add_assoc : ∀ a b c : R, a + b + c = a + (b + c))
#check (add_comm : ∀ a b : R, a + b = b + a)
#check (zero_add : ∀ a : R, 0 + a = a)
#check (neg_add_cancel : ∀ a : R, -a + a = 0)
-- 乘法
#check (mul_assoc : ∀ a b c : R, a * b * c = a * (b * c))
#check (mul_one : ∀ a : R, a * 1 = a)
#check (one_mul : ∀ a : R, 1 * a = a)
-- 分配律
#check (mul_add : ∀ a b c : R, a * (b + c) = a * b + a * c)
#check (add_mul : ∀ a b c : R, (a + b) * c = a * c + b * c)
```

这些事实既适用于抽象的 `R`, 也会自动适用于具体实例, 因为 Lean 会通过类型类机制自动找到相应的环实例。

ring 的使用场景 特别地, 当我们考虑交换环时(如上下文有 `[CommRing R]` 或 `[Field R]`) 时, 只要目标是“多项式恒等式”, 可以尝试 `ring` (它把等多项式暴力展开, 然后看长得一不一样):

```
section
variable (R : Type*) [CommRing R]
variable (a b : R)

-- 当你看到由 加、减、乘、乘方 组成的一团乱麻的等式, 并且知道变量满足交换律时, 敲 ring 就可以了。
example : (a + b) * (a + b) = a * a + 2 * (a * b) + b * b := by
  ring
end
```

直观上, 它相当于自动完成一长串 `rw [mul_add, add_mul, ...]` 的展开、合并同类项与重排。

常见套路： 由于 `ring` 只做代数恒等式的规范化，不会自动把你的假设“代入目标”。因此常见写法是先用 `rw` 把假设展开/代入，再用 `ring` 处理剩下的纯代数部分：

```
section
variable (R : Type*) [CommRing R]
variable (a b c d : R)

example (hyp : c = d * a + b) (hyp' : b = a * d) : c = 2 * a * d := by
  rw [hyp, hyp'] --  $\vdash d * a + a * d = 2 * a * d$ 
  ring
end
```

相关的代数自动化 `mathlib` 还提供了与 `ring` 思路类似的 `tactic`，用来覆盖更广的代数结构：

- `group`：用于乘法群（一般不交换）中的等式化简；
- `abel`：用于交换加法群中的等式化简；
- `noncomm_ring`：用于非交换环的等式化简¹（更一般，但通常更慢）。

3.2 不等式与格论

虽然 `rw` 和 `ring` 是证明等式的利器，但一旦目标变成不等式或更一般的序结构，原先那套“把表达式改写到一样”就不够用了。

本节给出两条主线：

- 在线性不等式上，利用 `linarith` 做一键自动化；
- 在更抽象的序结构上，学会把证明组织成“用通用接口引理推进”的套路（尤其是格论里的 `le_antisymm`、`le_inf`、`sup_le` 等）。

线性算术策略：linarith

当目标和假设属于**线性算术**（只包含加减、常数乘、 $\leq$ 、 $=$ 、 $<$ 等线性关系）时，`linarith` 通常是首选工具。可以把它理解成一个“线性约束求解器”：它会收集当前语境（Context）里的线性等式与不等式，把它们组合、消元、代入，尝试直接推出目标。对初学者而言，一个很实用的经验是：

如果你肉眼觉得“这题就是把几条线性不等式加起来/代一下就出结论”，那就先试 `linarith`。

```
variable (a b d : ℝ)

-- linarith 自动利用 h'' 将 d 替换为 2，并组合 h 与 h' 推出目标
example (h : 2 * a ≤ 3 * b) (h' : 1 ≤ a) (h'' : d = 2) : d + a ≤ 5 * b := by
  linarith
```

不适用场景： `linarith` 无法直接处理非线性项（如 a^2 、 $a \cdot b$ ）或超越函数（指数、对数等）。此时我们可以先把目标改写/降阶到线性问题。

¹命名上看起来有点“反直觉”，主要是历史原因；`ring` 使用最频繁，因此采用了更短的名字。

技巧：将“非线性”转化为“线性”

面对 `exp`、`log` 这样的非线性函数，我们可以

先用单调性/等价刻画把“函数不等式”剥离成“自变量不等式”，再用 `linarith` 收尾。

```
open Real
variable (a b : ℝ)

#check (exp_le_exp : exp a ≤ exp b ↔ a ≤ b)

example (h : a + 2 ≤ b) : exp (a + 1) ≤ exp b := by
  rw [exp_le_exp] -- ⊢ a + 1 ≤ b
  linarith
```

补充：非线性救星 `nlinarith`

如果你的目标中显式包含平方，且无法通过简单的重写消除，那么 `linarith` 就会失效。此时应使用 `nlinarith`，它自动利用了“实数平方非负”($x^2 \geq 0$) 这一隐含事实。但 `nlinarith` 并不擅长自动做因式分解。如果你给它一个展开的多项式（如 $a^2 - 2ab + b^2$ ），最好先手动将其整理为平方形式。

```
example (a b : ℝ) : a ^ 2 - 2 * a * b + b ^ 2 ≥ 0 := by
  -- 1. 先用 ring 帮我们把展开式整理回完全平方的形式
  have h : a ^ 2 - 2 * a * b + b ^ 2 = (a - b) ^ 2 := by ring
  -- 目标变成了 (a - b) ^ 2 ≥ 0
  nlinarith
```

3.2.1 格论 (Lattice)：统一的序结构接口

我们刚刚讨论了实数上的大小关系 ($\leq$)。但在数学中，“顺序”的概念无处不在：集合有包含关系，逻辑有蕴含关系，数论有整除关系。如果为每一种数学对象都单独定义一套引理（比如 `set.subset_trans`, `nat.dvd_trans`, `real.le_trans`），库会变得极其冗余。Mathlib 的解决方案是引入**格论 (Lattice Theory)**。

`Lattice` 类型类提供了一个统一的接口，它将“偏序关系”以及“最大下界 / 最小上界”抽象了出来：

- `PartialOrder` 给出偏序 ($\leq$ ，并带有反身、传递、反对称)；
- 偏序之上额外给出二元运算 `inf` ($\sqcap$) 与 `sup` ($\sqcup$)，分别表示**最大下界**与**最小上界**。

这一抽象的价值在于：一旦某个对象被证明是一个格，我们就能对它复用同一套“格论通用引理”：

- **全序集 (Total Orders)**：最直观的例子，如实数 $\mathbb{R}$ 。
 - $\sqcap$ (`meet`) $\rightarrow$ 取最小值 (`min`)
 - $\sqcup$ (`join`) $\rightarrow$ 取最大值 (`max`)
- **集合论**：全集的子集族，按 $\subseteq$ 排序。
 - $\sqcap \rightarrow$ 集合交集 ($\cap$)：两个集合的公共部分是它们最大的“下界”。
 - $\sqcup \rightarrow$ 集合并集 ($\cup$)：包含两个集合的最小集合是它们的“上界”。

- **数论**：自然数，按整除关系 $\mid$ 排序（注意不是大小！）。
 - $\sqcap \rightarrow$ 最大公约数 (gcd)：同时整除 a, b 的最大数。
 - $\sqcup \rightarrow$ 最小公倍数 (lcm)：同时被 a, b 整除的最小数。
- **逻辑**：命题或布尔值，按蕴含关系 $\rightarrow$ 排序。
 - $\sqcap \rightarrow$ 逻辑与 ($\wedge$)
 - $\sqcup \rightarrow$ 逻辑或 ($\vee$)
- **代数结构**：群的子群、向量空间的子空间等，通常按包含排序。
 - $\sqcap \rightarrow$ 子群的交集 (Interscetion)：两个子群的交依然是子群。
 - $\sqcup \rightarrow$ 子群的生成和 (Generated Subgroup)：两个子群的并集通常不是子群，所以上界是由并集生成的最小子群。

格论里的等式 在偏序结构里，想证等式 $u = v$ 时，不像算术那样能“移项/相消”。最稳妥的通用套路是把等式改写成“双向不等式”，也就是用**反对称性**：

$$u = v \iff (u \leq v) \text{ 且 } (v \leq u).$$

在 Mathlib 里这一步对应引理 `le_antisymm`。

接下来，“看到目标长什么样，就选对应的构造/消去引理”：

- 目标是 $X \leq a \sqcap b$ ：用 `le_inf`，把目标拆成 $X \leq a$ 和 $X \leq b$ 。
- （对偶地）目标是 $a \sqcup b \leq X$ ：用 `sup_le`，把目标拆成 $a \leq X$ 和 $b \leq X$ 。
- 目标是 $a \sqcap b \leq a$ 或 $a \sqcap b \leq b$ ：用投影公理 `inf_le_left` / `inf_le_right`。
- （对偶地）目标是 $a \leq a \sqcup b$ 或 $b \leq a \sqcup b$ ：用 `le_sup_left` / `le_sup_right`。

例 1：交运算交换律 $a \sqcap b = b \sqcap a$

```
-- α 是任意格
variable {α : Type*} [Lattice α] (a b c : α)

#check (le_antisymm : a ≤ b → b ≤ a → a = b)
#check (le_inf : c ≤ a → c ≤ b → c ≤ a ∩ b)
#check (inf_le_left : a ∩ b ≤ a)
#check (inf_le_right : a ∩ b ≤ b)

example : a ∩ b = b ∩ a := by
  apply le_antisymm
  • -- 证 a ∩ b ≤ b ∩ a: 右边是 inf, 用 le_inf 拆目标
    apply le_inf
    • exact inf_le_right -- a ∩ b ≤ b
    • exact inf_le_left  -- a ∩ b ≤ a
  • -- 对称地再来一遍
    apply le_inf
    • exact inf_le_right
    • exact inf_le_left
```

注：代码中的 `•` (点号) 用于隔离子目标的上下文，是一种良好的代码习惯。

例 2：吸收律 $a \sqcap (a \sqcup b) = a$

思路仍然是反对称性：一边用 “ $\sqcap$ 的投影”，另一边用 “要证 $\leq \text{inf}$ 就用 `le_inf`”。

```
-- 直观理解：a 和 (a ∪ b) 取交集，结果还是 a
variable {α : Type*} [Lattice α] (a b : α)

example : a ∩ (a ∪ b) = a := by
  apply le_antisymm
  • -- a ∩ (a ∪ b) ≤ a: inf 的左投影
    exact inf_le_left
  • -- a ≤ a ∩ (a ∪ b): 目标右边是 inf, 用 le_inf
    apply le_inf
    • exact le_rfl      -- a ≤ a
    • exact le_sup_left -- a ≤ a ∪ b
```

Chapter 4

结构

4.1 逻辑结构与证明策略

在 Lean 中，证明的“核心动作”往往不是计算，而是对目标（Goal）与假设（Context）里的逻辑结构进行拆解与构造。我们将按照“你面对的是目标还是假设”这一维度，将常用的逻辑连接词（ $\wedge, \vee, \forall, \exists, \rightarrow$ ）对应的 Tactic 进行分类。

4.1.1 第一类：当逻辑符号在“目标”中

当目标（ $\vdash$ 右侧）包含逻辑符号时，我们需要使用构造策略，将复杂目标拆解为简单的子目标。

1. 构造 $\wedge$ （与）：constructor

如果要证明 $P \wedge Q$ ，只需分别证明 P 和 Q 。

- **Tactic:** constructor
- **效果:** 生成两个子目标，一个证明左边，一个证明右边。

```
example (x y : ℝ) (h₀ : x ≤ y) (h₁ : x ≠ y) : x ≤ y ∧ x ≠ y := by
  constructor
  • exact h₀ -- 处理第一个分支
  • exact h₁ -- 处理第二个分支
```

2. 构造 $\vee$ （或）：left / right

如果要证明 $P \vee Q$ ，你必须做出选择：是证明 P 成立，还是证明 Q 成立。

- **Tactic:** left 或 right
- **效果:** 目标变为 P （如果选左）或 Q （如果选右）。

```
example (y : ℝ) (h : y > 0) : y > 0 ∨ y < -1 := by
  left -- 我们手里有 y > 0，所以选择证明左边
  exact h
```

3. 构造 $\rightarrow$ 或 $\forall$: intro

如果要证明“若 P 则 Q ”或“对任意 x ”，标准的数学动作是“假设 P 成立”或“任取一个 x ”。

- **Tactic:** `intro` 变量名 / 假设名
- **效果:** 将前提或变量移入 Context（假设区），目标变为结论部分。

```
-- 目标:  $\forall a\ b, a \leq b \rightarrow f\ a \leq f\ b$ 
example (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :  $\forall a\ b, a \leq b \rightarrow f\ a \leq f\ b$  := by
  intro a b h_le -- 引入变量 a, b 和假设 h_le
  -- 现在的目标变成了:  $f\ a \leq f\ b$ 
  sorry
```

4. 构造 $\exists$ (存在): use

要证明“存在一个 x 满足 $P(x)$ ”，你需要给出一个具体的例子（Witness）。

- **Tactic:** `use` 值
- **效果:** 将目标中的存在量词剥离，变为证明该值满足性质。

```
example :  $\exists x : \mathbb{R}, x > 10$  := by
  use 100
  -- 现在的目标变成了:  $100 > 10$ 
  norm_num
```

4.1.2 第二类：当逻辑符号在“假设”中

当假设（Context）中包含复杂的逻辑命题（如 $h : P \vee Q$ ）时，我们需要使用拆解策略（Destruction）来提取信息。最通用的拆解工具是 `rcases`（Recursive Cases）。

1. 拆解 $\wedge$ (与) 与 $\exists$ (存在)

如果我们知道“ P 且 Q ”，或者“存在一个 x 满足 P ”，这意味着我们同时拥有了两样东西（两个事实，或者一个值加一个事实）。

- **语法:** `rcases h with <名字 1, 名字 2>`
- **效果:** 将假设 h 拆成两部分。

```
example (h :  $\exists x, x > 5 \wedge x < 10$ ) : True := by
  -- 将 h 拆解为一个具体的值 x₀, 以及两个性质
  rcases h with <x₀, <h_gt, h_lt>>
  -- 现在 Context 里有了:
  -- x₀ :  $\mathbb{R}$ 
  -- h_gt :  $x_0 > 5$ 
  -- h_lt :  $x_0 < 10$ 
  trivial
```

2. 拆解 $\vee$ (或): 分类讨论

如果我们知道 “ P 或 Q ”，则必须进行分类讨论。

- 语法: `rcases h with h_left | h_right`
- 效果: 生成两个分支。分支一假设 P 成立 (命名为 `h_left`)；分支二假设 Q 成立 (命名为 `h_right`)。

```
example (x : ℝ) (h : x = 0 ∨ x = 1) : x * (x - 1) = 0 := by
  rcases h with h0 | h1
  • -- 分支 1: 假设 x = 0 (h0)
    rw [h0]; norm_num
  • -- 分支 2: 假设 x = 1 (h1)
    rw [h1]; norm_num
```

4.1.3 逻辑变换与自动化

除了手动的构造与拆解，我们还需要处理 “否定” 以及使用自动化工具来清理琐碎的逻辑步骤。

1. 处理否定 ($\neg$): `push_neg`

在数学中，我们经常需要把 $\neg(\forall x, Px)$ 改写为 $\exists x, \neg Px$ 。手动做这些变换很痛苦，Lean 提供了 `push_neg`。

- `push_neg`: 作用于目标，将否定号推入命题最深处。
- `push_neg at h`: 作用于假设 h 。

```
example (h : ¬ ∀ x, x > 0) : ∃ x, x ≤ 0 := by
  -- h : ¬ ∀ (x : ℝ), x > 0
  push_neg at h
  -- 现在 h 变成了 : ∃ x, x ≤ 0
  exact h
```

2. 数值计算自动化: `norm_num`

我们在前文介绍了处理代数的 `ring` 和处理不等式的 `linarith`。这里补充一个专门处理 “具体数字” 的工具。`norm_num` 擅长判断如 $2 + 2 = 4$ 、 $(3 :) < 5$ 这类纯数值问题。

```
example : (1 : ℝ) + 2 < 4 := by
  norm_num -- 自动计算并验证
```

3. 纯逻辑自动化: `tauto`

如果一个命题仅仅是逻辑上的重言式 (Tautology)，不涉及具体的数学计算 (如加减乘除)，可以直接交给 `tauto`。它就像是自动化真值表。

```
example (p q : Prop) : p ∧ q → p ∨ q := by
  tauto -- 逻辑真理，无需废话
```

4.1.4 总结：逻辑操作速查表

逻辑符号	在目标中 (构造)	在假设中 (拆解)
$\wedge$ (与)	constructor	rcases h with ⟨h1, h2⟩
$\vee$ (或)	left / right	rcases h with h1 h2
$\rightarrow$ (蕴含)	intro h	apply h / have h2 := h h1
$\forall$ (全称)	intro x	specialize h x / apply h
$\exists$ (存在)	use value	rcases h with ⟨value, h_prop⟩
$\neg$ (非)	intro / push_neg	push_neg at h

4.1.5 自动化简化：simp

`simp` 是 Lean 中最常用的自动化战术之一。如果说 `rw` 是手动的“精准手术”，那么 `simp` 就像是一个“智能吸尘器”：它会自动在库中寻找并应用那些可以将表达式变得更简单的引理。

工作原理：规范化 (Canonicalization)

直觉上，`simp` 的目标是将表达式规范化。它反复应用重写规则，直到目标无法被进一步简化为止。常见的简化包括：

- 算术化简： $x + 0 \rightarrow x, x * 1 \rightarrow x$
- 集合论化简： $x \in s \cap t \rightarrow x \in s \wedge x \in t$
- 逻辑化简： $p \wedge \text{True} \rightarrow p$

自定义简化规则：@[simp]

你可能会问：`simp` 怎么知道哪些引理可以用来化简？答案是：`simp` 维护了一个巨大的数据库。任何标有 `@[simp]` 属性的引理都会被自动加入这个数据库。

你自己也可以定义 `simp` 引理！当你定义了一个新函数后，通常应该立即证明其在基础值上的行为，并标记为 `@[simp]`，这样以后 `simp` 就能自动处理这个函数了。

```
-- 1. 定义一个新函数
def double (n : Nat) : Nat := n + n

-- 2. 告诉 simp 如何化简这个函数的一个特例
-- 使用 @[simp] 属性标记此引理
@[simp] lemma double_zero : double 0 = 0 := rfl

-- 3. 实战：simp 现在变聪明了
example (x : Nat) : double 0 + x = x := by
  -- simp 会自动找到 double_zero, 把 double 0 变成 0
  -- 然后利用库里的 add_zero 把 0 + x 变成 x
  simp
```

工程最佳实践: simp only

虽然直接输入 `simp` 很爽，但在大型项目（如 Mathlib）中，我们强烈建议使用 `simp only [...]` 显式指定要用的引理。

- **稳定性:** 如果未来库里新增了一个 `simp` 引理，可能会改变简化路径，导致你原本能跑通的 `simp` 突然挂掉。使用 `simp only` 可以由你完全掌控局面。
- **速度:** `simp` 需要搜索成千上万条引理，而 `simp only` 只需要查表几次。

```
example {α : Type} {s t : Set α} {x : α} : x ∈ s ∪ t ↔ x ∈ s ∨ x ∈ t := by
  simp only [Set.mem_union]
```

探查利器: simp?

- **功能:** 它像 `simp` 一样工作，但会在 InfoView 中打印出它具体使用的引理列表。
- **用法:** 先写 `simp?`，然后点击 InfoView 中的 **Try this: ...** 链接，VS Code 会自动把你的代码替换成稳健的 `simp only [...]` 写法。

```
example (n : Nat) : n + 0 = n := by
  -- 1. 你想化简，但不知道具体引理叫 add_zero 还是 zero_add
  -- 于是你写:
  simp?
  -- 2. Lean 在右侧 InfoView 告诉你:
  -- "Try this: simp only [add_zero]"
  -- 3. 你点击那个蓝色链接，代码自动变为:
  simp only [add_zero]
```

总结流程: 开发时用 `simp?` 探路 → 点击替换为 `simp only` → 提交代码。这样既享受了自动化的便利，又保证了代码的健壮性。

位置修饰

同 `rw` 一样，`simp` 也可以指定作用域：

- `simp at h`: 仅化简假设 `h`。
- `simp at *`: 同时化简所有假设和目标（威力巨大，但容易把上下文搞乱，调试时慎用）。

```
example (x y : Nat) (h : x + 0 = y) : x = y := by
  simp at h
  exact h
```

4.1.6 结构化证明: calc 模式

当证明需要进行一连串的代数变形或不等式放缩时，传统的 `Tactic` 模式（堆砌 `rw` 或 `apply`）往往会让人迷失方向。`calc` 模式旨在模仿人类在黑板上写数学推导的格式：一步一个台阶，每一步都给出理由。

基本语法

一个 `calc` 块由多行组成，每一行都遵循 表达式 + 关系符 + 表达式 + 证明的结构：

```
calc
  A = B := by proof1
  _ = C := by proof2
  _ = D := by proof3
```

- 下划线 (`_`)：这是一个占位符，代表上一行的右端。这省去了重复抄写长表达式的麻烦。
- 传递性：Lean 会自动利用关系的传递性 (Transitivity)。如果证明了 $A = B, B = C, C = D$ ，Lean 自动推出 $A = D$ 。

示例 1：纯等式推导

我们来证明 $(a + b)^2 = a^2 + 2ab + b^2$ 。虽然用 `ring` 可以秒杀，但用 `calc` 可以展示推导细节：

```
example (a b : ℝ) : (a + b)^2 = a^2 + 2 * a * b + b^2 := by
  calc
    (a + b)^2
      = (a + b) * (a + b) := by rw [pow_two]
    _ = a * (a + b) + b * (a + b) := by rw [add_mul]
    _ = a * a + a * b + (b * a + b * b) := by rw [mul_add, mul_add]
    _ = a^2 + 2 * a * b + b^2 := by ring
```

示例 2：混合关系的链式推导

`calc` 强大的地方在于它可以混合使用不同的关系符 (如 $=, \leq, <$)，只要这些关系在逻辑上能连通。Lean 会自动处理这种“混合传递性”：

$$(A = B) \wedge (B < C) \wedge (C \leq D) \implies A < D$$

```
-- 假设我们已知以下事实
variable (a b c d : ℝ)
variable (h1 : a = b) (h2 : b < c + 1) (h3 : c + 1 ≤ d)

example : a < d := by
  calc
    a = b      := h1      -- 利用等式
    _ < c + 1   := h2      -- 利用小于
    _ ≤ d      := h3      -- 利用小于等于
  -- Lean 自动推断出结论：a < d
```

技巧：配合 `ring` 使用 在处理不等式时，我们经常需要在某一步进行纯代数变形 (等式)，然后再进行放缩。可以在 `calc` 中随意穿插 `by ring` 来处理繁琐的代数整理。

```
example (n : ℕ) : (n + 1)^2 ≥ 4 * n := by
  calc
```

```

(n + 1)^2
  = n^2 + 2 * n + 1 := by ring
_ ≥ 2 * n + 2 * n   := by nlinarith -- 即使中间一步跳跃较大也没关系
_ = 4 * n           := by ring

```

实践建议

- 逻辑结构优先“结构化处理”（`constructor/intro/rcases`），再考虑自动化（`simp/linarith/tauto`）。
- 自动化优先“可控”（`simp only [...]`、先把目标整理成线性再 `linarith`），避免一次性把 proof state 变得难以预测。
- `calc` 适合“主干推导清晰”的证明；局部技巧（化简、重写、自动化）尽量放在每一步的 `by ...` 里，让结构保持可读。

Chapter 5

Mathlib 指南

5.1 寻找 Mathlib 定理

写证明时，最大的瓶颈往往不是 tactic，而是：你数学上知道要用什么定理，但不知道它在 mathlib 里叫什么/在哪个文件里。下面给出一套“从快到慢”的找法（优先使用前两条）：

- （最常用）靠命名习惯 + 自动补全：mathlib 的定理名高度规律化。比如想找形如 $x + y \leq \dots$ 的引理，通常以 `add_le_` 开头；与指数有关常见是 `exp_`；与最小值有关常见是 `min_` 等。在编辑器中输入前缀后用自动补全（Ctrl-Space / Cmd-Space）往往能直接命中。
- （非常实用）`#search / apply?`：当你已经写出了目标的形状时，让 Lean 帮你找。

```
-- 例：让 Lean 猜一个能证明  $0 \leq a^2$  的引理
example (a : ℝ) : 0 ≤ a ^ 2 := by
  apply?
-- 常见结果会提示类似 sq_nonneg a 或 pow_two_nonneg a 等
```

- 文档页与源码：
 - mathlib API 文档：https://leanprover-community.github.io/mathlib4_docs/
 - theories 总览页（按领域导览）：<https://leanprover-community.github.io/theories/>
 - GitHub 源码：<https://github.com/leanprover-community/mathlib4>
- 专用搜索工具（按模式搜类型）¹：
 - Loogle：<https://loogle.lean-lang.org/>（vscode 里直接右键也行）
 - leansearch：<https://leansearch.net/>（also try `#search`）
 - LeanExplore：<https://www.leanexplore.com/>
- 从一个已知定理“就地扩展”：在 VS Code 里右键已有定理名，跳转到定义文件；同一文件附近往往有一整组相似引理（例如一串 `add_le_add_*`、`min_le_*`）。

实践建议：先用“命名习惯 + 自动补全”猜一个候选名；不行就用 `#search / apply?`；再不行才去文档页/源码/外部工具。

¹note their mathlib versions

5.2 Lean 中的命名法

为了让代码更接近主流编程语言、并且一眼区分“可运行的程序/数据”和“需要证明的命题/定理”，Lean（尤其是 mathlib 社区）约定了一套比较严格的命名规范。本节内容主要参考 mathlib 的命名指南：<https://leanprover-community.github.io/contribute/naming.html>。

Lean 中最常见的三种命名风格是：

- snake_case: 全小写，用下划线连接单词；
- lowerCamelCase: 第一个单词小写，其后每个单词首字母大写，不使用分隔符；
- UpperCamelCase: 每个单词首字母大写，不使用分隔符。

核心规则（记住这几条就够用）

1. 命题（Prop）相关：用 snake_case。

典型包括：定理名、引理名、证明名（即类型为 Prop 的项）。

例：nat.add_comm、mul_assoc、map_one。

2. 类型/结构：用 UpperCamelCase。

典型包括：归纳类型、structure、class（以及它们的名字）。

例：Nat、List、MonoidHom、TopologicalSpace。

3. 普通函数与数据：用 lowerCamelCase。

也就是既不是类型名、也不是命题/定理名的“普通项”，例如一般函数、构造出的数据等。

例：toFun、mapMul、simpLemma（示意）。

4. 函数名通常“跟随返回值的风格”。

直觉上：一个东西“最终返回什么类型”，它的名字往往就采用那个层级应当使用的风格。例如某个字段返回 Prop，那字段名往往是 snake_case；返回普通数据则常用 lowerCamelCase。

两个常见细节

- 在 snake_case 里提到 UpperCamelCase 的类型名：通常把该类型名改成 lowerCamelCase 形式嵌入。例如类型叫 MyType，相关引理可能写成 myType_eq_foo 之类的形式。
- 缩写（Acronym）：像 LE、LT 这类缩写一般作为整体处理，具体大小写以首字符为准；mathlib 中也存在少量历史例外（属于“已存在代码的兼容”）。

例子

```
-- 类型/结构: UpperCamelCase
structure OneHom (M : Type _) (N : Type _) [One M] [One N] where
  toFun : M → N           -- 普通字段: lowerCamelCase
  map_one' : toFun 1 = 1 -- 命题字段: snake_case (结尾 ' 常用于避免重名)

-- 定理/引理: snake_case
theorem map_one [OneHomClass F M N] (f : F) : f 1 = 1 := sorry
```

定理名怎么起：结论优先，条件用 of 串起来

实践中你最常需要处理的是定理命名。mathlib 的风格通常是：

结论描述 of 条件 1 of 条件 2 ...

也就是说：先把结论写在名字里，再用 of 把关键假设串起来，让人只看名字就能大致猜到定理在说什么。

例子

```
#check lt_of_le_of_ne
-- 输出签名:  $\forall \{a\ b : \alpha\}, a \leq b \rightarrow a \neq b \rightarrow a < b$ 

-- 命名解析
-- 1. lt (Less Than)      : 结论是  $a < b$ 
-- 2. le (Less Equal)     : 条件1是  $a \leq b$ 
-- 3. ne (Not Equal)      : 条件2是  $a \neq b$ 
```

对于极少数“历史上约定俗成、且大家都认识”的大定理，mathlib 也会使用更接近传统名称的命名方式（比如 `Cauchy-Schwarz-Inequality`）；但总体上仍会尽量保持可搜索、可读、可预测的风格（尤其在 API 规模很大时，这一点非常重要）。

后续写作业/项目时，建议你尽量遵循上述规则：这样不仅更符合社区习惯，也更利于自动补全、搜索、以及与他人协作（包括 AI 工具的检索与调用）。

5.2.1 常见符号与缩写速查表

Mathlib 的定理命名高度依赖于符号的英文缩写。熟练掌握下表中的对应关系，能够帮助你快速通过名称推测定理内容，或通过内容反推定理名称。

1. 逻辑与集合符号

符号	输入代码	命名片段	说明与备注
$\vee$	<code>\or</code>	<code>or</code>	
$\wedge$	<code>\and</code>	<code>and</code>	
$\rightarrow$	<code>\r</code>	<code>imp / of</code>	<code>imp</code> 指命题本身, <code>of</code> 指该命题作为前提
$\leftrightarrow$	<code>\iff</code>	<code>iff</code>	常省略, 尤其是定义等价关系时
$\neg$	<code>\n</code>	<code>not</code>	
$\exists$	<code>\ex</code>	<code>exists</code>	<code>bex</code> (Bounded Exists) 在 Mathlib 中较少直接用于命名
$\forall$	<code>\fo</code>	<code>forall / all</code>	<code>ball</code> (Bounded All) 同上, 多用于 Lean Core
$=$		<code>eq</code>	在定理名中经常省略
$\neq$	<code>\ne</code>	<code>ne</code>	
$\circ$	<code>\o</code>	<code>comp</code>	函数组合 (Composition)
$\in$	<code>\in</code>	<code>mem</code>	成员关系 (Member)
$\cup$	<code>\cup</code>	<code>union</code>	二元并集
$\cap$	<code>\cap</code>	<code>inter</code>	二元交集
$\bigcup$	<code>\bigcup</code>	<code>iUnion</code>	索引并集 (Indexed Union)
$\bigcap$	<code>\bigcap</code>	<code>iInter</code>	索引交集 (Indexed Intersection)
$\bigcup_0$	<code>\bigcup\0</code>	<code>sUnion</code>	集合的并集 ($\bigcup S$)
$\bigcap_0$	<code>\bigcap\0</code>	<code>sInter</code>	集合的交集 ($\bigcap S$)
$\setminus$	<code>\</code>	<code>sdiff</code>	集合差 (Set Difference)
$\{x \mid px\}$		<code>setOf</code>	集合构造
$\{x\}$		<code>singleton</code>	单元素集
$\{x, y\}$		<code>pair</code>	对集

2. 代数与运算符号

符号	输入代码	命名片段	说明与备注
0		<code>zero</code>	
1		<code>one</code>	
$+$		<code>add</code>	
$-$		<code>neg / sub</code>	<code>neg</code> 为取负 ($-a$), <code>sub</code> 为减法 ($a - b$)
$*$		<code>mul</code>	一般乘法
$\cdot$	<code>\bu</code>	<code>smul</code>	数乘 (Scalar Multiplication)
$^$		<code>pow</code>	幂运算
$/$		<code>div</code>	除法
-1	<code>\-1</code>	<code>inv</code>	群逆元
$\frac{1}{\cdot}$	<code>\frac1</code>	<code>invOf</code>	环中的单位元逆
$ $	<code>\ </code>	<code>dvd</code>	整除 (Divides)
$\sum$	<code>\sum</code>	<code>sum</code>	有限和
$\prod$	<code>\prod</code>	<code>prod</code>	有限积

3. 序与格论符号

重要惯例：在 Mathlib 中，不等式几乎总是优先使用 $\leq$ (`le`) 和 $<$ (`lt`)。即便你想表达“大于等于”，也通常会找到名为 `le` 的定理（例如 $b \geq a$ 会被写成 $a \leq b$ 处理）。

符号	输入代码	命名片段	说明与备注
$<$		<code>lt</code>	Less Than
$\leq$	<code>\le</code>	<code>le</code>	Less Equal
$\sqcup$	<code>\sup</code>	<code>sup</code>	上确界/并 (二元)
$\sqcap$	<code>\inf</code>	<code>inf</code>	下确界/交 (二元)
$\sqcup$	<code>\sqcup</code>	<code>iSup</code> / <code>ciSup</code>	<code>c</code> 表示条件完备 (Conditionally Complete)
$\sqcap$	<code>\sqcap</code>	<code>iInf</code> / <code>ciInf</code>	同上
$\perp$	<code>\bot</code>	<code>bot</code>	底元素 (Bottom)
$\top$	<code>\top</code>	<code>top</code>	顶元素 (Top)

4. 拼写惯例 (Spelling Conventions)

Mathlib 严格遵循美式英语拼写。

- 使用 **z** 而非 **s**: `factorization`, `localization` (勿用 *factorisation*)。
- 使用 **er** 而非 **re**: `fiber`, `center` (勿用 *fibres*, *centre*)。

5.2.2 命名空间 (Namespaces)

在数学形式化过程中，我们经常遇到名称冲突的问题。例如，自然数 $\mathbb{N}$ 、实数 $\mathbb{R}$ 以及格结构中都有“最小值”的概念，如果我们都将其命名为 `min_le_right`，系统将无法区分。

为了解决这个问题，Lean 引入了命名空间 (Namespace) 机制。

- 定义：**使用 `namespace` 关键字可以将一系列定义包裹在一个特定的作用域内。
- 效果：**在命名空间内定义的定理，其全名会自动加上空间名前缀。
- 调用：**在空间外调用时，需使用“点号表示法”（如 `Nat.add`）；或者使用 `open` 关键字打开空间。

```
-- 开启名为 my_theory 的命名空间
namespace my_theory

-- 定义一个定理，全名实际上是 my_theory.and_of_left_of_right
theorem and_of_left_of_right {p q : Prop} (hp : p) (hq : q) : p ∧ q :=
  And.intro hp hq

end my_theory -- 结束命名空间

-- 在外部调用时，必须加前缀
#check my_theory.and_of_left_of_right

-- 或者使用 open 打开它，之后便可直接使用
open my_theory
#check and_of_left_of_right
```

最佳实践: Mathlib 通常通过命名空间来组织不同数学结构的定理。例如, 关于列表的定理都在 `List` 空间下, 关于自然数的在 `Nat` 空间下。这也是为什么你会看到 `List.map` 和 `Option.map` 这类重名函数能和平共处的原因。

5.3 Mathlib 代码风格指南

Lean 社区维护了一套严格的风格指南 (Style Guide) 参考 <https://leanprover-community.github.io/contribute/style.html>, 这不仅是为了美观, 更是为了保证代码的可读性、可维护性以及编译器的稳定性。除了命名规范外, 以下几个方面是编写高质量 Lean 代码的关键。

5.3.1 排版与空白 (Layout & Formatting)

代码的视觉布局应清晰且一致。虽然 Lean 对空白不敏感, 但人类读者敏感。

- **行宽限制:** 每行代码不应超过 **100 个字符**。过长的行应在适当的运算符处换行。
- **缩进:**
 - 必须使用 **2 个空格** 进行缩进。
 - 严禁使用 Tab 字符。
- **空格的使用:**
 - 二元运算符前后需加空格。
 - 冒号 (`:`)、赋值符 (`:=`) 前后需加空格。
 - 逗号 (`,`) 后需加空格。
 - 括号内侧不加空格, 函数调用时不加空格。

```
-- [错误示范] 拥挤、无空格、括号内有多余空格
def f(x:Nat):Nat:=x+1
#check ( f 5 )

-- [正确示范] 清晰的间隔
def f (x : Nat) : Nat := x + 1
#check f 5
```

5.3.2 显式与隐式参数 (Explicit vs Implicit Arguments)

在定义引理或函数时, 参数括号的选择 (`()`, `{}`, `[]`) 至关重要。

- **隐式参数 { }:** 如果一个参数可以通过后续参数或返回类型被推断出来, 应设为隐式。
- **显式参数 ():** 如果一个参数无法被推断, 或者经常需要用户手动提供, 应设为显式。
- **类型类参数 []:** 用于 `Group`, `Ring`, `Fintype` 等结构。

```

variable {G : Type*} [Group G]

-- [错误示范] G 和 x 都能被推断，不应显式传入
-- 用户调用时被迫写 mul_inv_self G x，非常繁琐
theorem mul_inv_self_bad (G : Type*) [Group G] (x : G) : x * x⁻¹ = 1 := ...

-- [正确示范] G 和群结构自动推断，x 由上下文决定
-- 用户调用时只需写 mul_inv_self x (甚至只写 mul_inv_self)
theorem mul_inv_self {G : Type*} [Group G] (x : G) : x * x⁻¹ = 1 := ...

```

5.3.3 证明风格 (Proof Style)

非终结 Simp (Non-terminal Simp)

这是 Mathlib 中最严格的禁忌之一。不要在证明中间使用不带参数的 `simp`。

- **原因：**`simp` 集合随着库的更新不断扩大。今天能跑通的 `simp`，明天可能会因为多引入了一个引理而把目标化简成不同的样子，导致后续的 `rw` 或 `exact` 失败（代码易碎）。
- **例外：**如果 `simp` 能直接闭合目标（即证明结束），则是允许的。
- **修正：**使用 `simp only [...]` 明确指定要用的引理，或者使用 `squeeze_simp` 辅助生成。

```

-- [脆弱的风格]
example : A = B := by
  simp          -- 危险！如果 simp 行为改变，下面这行可能会挂
  rw [h]

-- [推荐的风格]
example : A = B := by
  simp only [add_comm, add_zero] -- 明确指定，稳如泰山
  rw [h]

```

结构化证明 (Structured Proofs)

避免写一长串难以理解的 tactic 链。优先使用 `have`, `let`, `calc` 等结构化工具，使证明过程对人类可读。

```

-- [难以阅读] 只有机器知道中间发生了什么
example : a * b * c = d := by
  rw [mul_assoc, mul_comm b, ←mul_assoc, h1, h2, h3]

-- [清晰明了] calc 块展示了推导思路
example : a * b * c = d := by
  calc a * b * c
    _ = a * (b * c) := by rw [mul_assoc]
    _ = a * d       := by rw [h_bc_eq_d]
    _ = d           := by rw [h_ad_eq_d]

```

5.3.4 文档规范 (Documentation)

代码不仅仅是写给编译器看的。

- **模块文档**: 每个 Lean 文件的开头必须包含模块级文档, 描述该文件的数学定义、主要定理和符号约定。
- **文档字符串 (Docstrings)**: 每个顶层定义 (`def`, `lemma`, `structure`) 上方都应添加文档, 使用 `/-- ... -/` 格式。
- **Markdown 支持**: 文档中可以使用 Markdown 语法 (如反引号引用代码) 和 LaTeX 公式。

```
/--  
如果一个群元素的平方是 1, 则它是自己的逆。  
注意: 这是一个简单的示例引理。  
-/  
lemma inv_eq_self_of_sq_eq_one {x : G} (h : x * x = 1) : x⁻¹ = x := by  
  ...
```

Chapter 6

类型的层级

6.1 Universe

6.1.1 类型的类型

在前文中，我们已经学会了使用 `#check` 命令来查看一个项 (Term) 的类型：

```
#check 1 -- 1 : ℕ
```

既然 $1 : \mathbb{N}$ ，那么 $\mathbb{N}$ 本身也是一个对象，它的类型又是什么？同样的， $\mathbb{R}$ 和 `Prop` 的类型是什么？

```
#check ℕ -- ℕ : Type
#check ℝ -- ℝ : Type
#check Prop -- Prop : Type
```

结果显示： $\mathbb{N}$ 、 $\mathbb{R}$ 这些“数学对象”在 Lean 中本身也是类型，而它们都属于同一个层级：`Type`。

6.1.2 宇宙层级

如果我们追问 `Type` 的类型是什么，会发现一个有趣的现象：Lean 不允许 `Type : Type` 这种循环包含。如果允许类型包含自身，将会导致类似集合论中“罗素悖论”的逻辑矛盾。

为了保证逻辑一致性，Lean 引入了**宇宙层级 (Universe)** 的概念，将类型划分为无穷向上的层级：

$$\text{Type } 0 : \text{Type } 1 : \text{Type } 2 : \dots$$

我们在代码中常见的 `Type`，实际上只是 `Type 0` 的语法糖。为了在不同层级间复用代码（例如定义一个既能处理 `Type 0` 又能处理 `Type 1` 的列表），我们便需要引入**宇宙变量**（如常见的 `universe u`）。

Sort：逻辑与数据的统一

在更底层的实现中，Lean 将表示逻辑命题的 `Prop` 和表示数据的 `Type` 统一在了 `Sort` 体系之下。它们的对应关系如下：

$$\text{Prop} \equiv \text{Sort } 0, \quad \text{Type } u \equiv \text{Sort } (u+1)$$

这意味着 `Prop` 和 `Type` 只是 `Sort` 家族在不同层级上的成员。`Sort 0` 是命题的世界，而 `Sort 1` 及以上则是数据类型的世界。我们可以通过以下代码来验证这种等价关系：

```
-- Type 的层级结构
#check Type      -- Type : Type 1
#check Type 1    -- Type 1 : Type 2

-- Prop 和 Type 在 Sort 体系中的位置
#check Sort 0    -- Prop : Type
#check Sort 1    -- Type : Type 1
#check Sort 2    -- Type 1 : Type 2

-- universe 变量实际上是 Sort 的参数
universe u
#check Sort u     -- Sort u : Type u
#check Sort (u+1) -- Type u : Type (u + 1)
```

6.2 核心机制：函数类型

在类型论中，函数类型是构建一切复杂结构（包括逻辑蕴含和全称量词）的基石。

6.2.1 一元函数与 λ 构造

给定任意两个类型 α 与 β ，我们可以构造出函数类型 $\alpha \rightarrow \beta$ 。

- 这是一个“从输入 α 得到输出 β ”的程序。

要构造一个函数，我们使用关键字 `fun` 或符号 λ 。以下三种写法是完全等价的：

```
variable (A B : Type) (f : A → B) (a : A)
#check f a      -- 函数应用不需要括号，使用空格左结合

-- 1. 标准写法
def f1 : Nat → Nat := fun n => n + 1
-- 2. 使用 lambda 符号
def f2 : Nat → Nat := λ n => n + 1
-- 3. 使用 mapsto 箭头
def f3 : Nat → Nat := fun n ↦ n + 1
```

6.2.2 多元函数的本质：柯里化 (Currying)

在 Lean 中，本质上不存在“多元函数”。所有的多元函数实际上都是通过柯里化实现的：一个接受两个参数的函数，本质上是一个“接受第一个参数，返回一个‘接受第二个参数并返回结果的函数’的函数”。

理解这一点的关键在于掌握两个相反的结合规则：

- 类型构造是右结合的： $A \rightarrow B \rightarrow C$ 等价于 $A \rightarrow (B \rightarrow C)$ 。
- 函数应用是左结合的： $f\ a\ b$ 等价于 $(f\ a)\ b$ 。

虽然底层是嵌套的 λ 表达式，但 Lean 提供了语法糖，允许我们将多个参数写在同一个 `fun` 后面：

```
-- 语法糖写法 (推荐)
def add_simple : Nat → Nat → Nat := fun n m => n + m
```

6.2.3 依赖函数类型 (Π -type)

依赖函数是普通函数类型的推广。在普通函数 $A \rightarrow B$ 中, 返回值类型 B 是固定的。而在依赖函数中, 返回值的类型可以依赖于输入参数的值。

设 $A : \text{Type}$, $B : A \rightarrow \text{Type}$ 是一个类型族。对于输入参数 $a : A$, 依赖函数的返回值类型为 $B\ a$ 。这种类型记作 $(a : A) \rightarrow B\ a$, 在 Mathlib 中常写作 $\Pi\ a : A, B\ a$ 。

```
variable (A : Type) (B : A → Type)
-- f 是一个依赖函数: 给它不同的 a, 返回值的类型 B a 也会随之变化
variable (f : (a : A) → B a)
```

当类型不依赖参数时

依赖函数类型 $\Pi x : A, B(x)$ 涵盖了普通函数类型: 如果返回值的类型 B 并不实际依赖于输入 x (即 B 是个常数类型), 那么它就退化成了我们熟悉的普通函数箭头 $\rightarrow$ 。

```
universe u v
variable (A : Type u) (C : Type v)

-- 当 B(x) 恒等于 C 时:
-- ( $\Pi\ x : A, C$ ) 等价于  $(A \rightarrow C)$ 
-- 下划线 _ 表示我们忽略了输入参数 x, 因为它不影响返回类型
example : ( $\Pi\ _ : A, C$ ) = (A → C) := rfl
```

全称量词: 命题世界里的 Π

在逻辑中, $\forall x : A, P(x)$ 意味着“对于任意的 x , $P(x)$ 都成立”。在 Lean 中, 这被实现为一个依赖函数: 你给我一个 x , 我通过函数返回给你一个 $P(x)$ 的证明。

```
-- 命题: 对于任意自然数 n, n 等于 n
def forall_example :  $\forall\ n : \text{Nat}, n = n$  :=
  -- 证明就是一个函数:
  -- 输入是 n (自然数)
  -- 输出是 rfl (证明  $n = n$  的项)
  fun n => rfl
```

6.3 逻辑视角: 命题即类型

有了上述函数工具, 我们就可以理解类型论中最核心的思想——Curry-Howard 同构 (Propositions as Types)。

6.3.1 蕴含即函数 ($\rightarrow$)

在 Lean 中, 逻辑蕴含 $p \rightarrow q$ 实际上就是函数类型。

- **参数即假设:** 函数的输入参数, 对应定理的已知条件。
- **返回值即结论:** 函数的返回值, 对应定理要证明的目标。

这就是为什么同一个定理可以写得像 C++ 函数, 也可以写得像数学公式:

```
-- 写法 1: 函数式风格
theorem logic_func {p q : Prop} (hp : p) (hq : q) : p ∧ q :=
  sorry

-- 写法 2: 逻辑式风格 (使用 →)
theorem logic_arrow : ∀ {p q : Prop}, p → q → p ∧ q :=
  sorry
```

6.4 证明: Term vs Tactic

既然定理是类型，证明就是构造该类型的项 (Term)。我们有两种构造方式。

6.4.1 Term Proof: 直接构造

当证明逻辑简单 (如函数组合) 时，我们可以直接写出那个项。这就像在搭积木：

```
variable (p q r : Prop)
-- 逻辑三段论 = 函数组合
example (h1 : p → q) (h2 : q → r) (hp : p) : r :=
  h2 (h1 hp)
```

解析： h_1 hp 产生了一个类型为 q 的项，再把它喂给 h_2 ，最终得到类型为 r 的项。

6.4.2 Tactic Proof: 指令式构造

当证明涉及复杂的代数变形时，直接写出 Term 会极其痛苦。Tactic 模式允许我们使用指令 (如 `rw`, `ring`) 来逐步改变目标。

```
-- 痛苦的 Term 写法 (仅作反面教材)
example (a b c : R) : a * b * c = c * b * a :=
  (mul_assoc a b c).symm ▸ (mul_comm c (a * b)) ▸ (mul_assoc c a b) ▸ ...

-- 清晰的 Tactic 写法
example (a b c : R) : a * b * c = c * b * a := by
  ring
```

6.4.3 show_term: 揭示真相

Tactic 并不是魔法，它们只是帮你写代码的“宏”。最终它们都会被编译器翻译成一个底层的项。使用 `show_term` 可以查看 Tactic 背后生成的复杂 Term：

```
variable (f : R → R) (a b : R)

example (h : a = b) : f a = f b := by
  show_term {
    rw [h]
  }
-- InfoView 输出:
```

```
-- exact Eq.mpr (id (congrArg (fun _a => f _a = f b) h)) (Eq.refl (f b))
```

解析：你会发现，简单的 `rw [h]` 实际上构造了一个涉及 `Eq.mpr`（利用等式改写类型）和 `congrArg`（函数的参数全等性）的复杂函数调用。这就是 `Tactic` 存在的意义：把人类从繁琐的底层构造中解放出来。

6.5 Exercises

6.5.1 命名

请为以下的 examples 起一个合乎规范的名字（其实你们多数也见过了）：

```
variable {α : Type}

example {a : ℝ} : a + 0 = 0 := by sorry

example {a b c : ℝ} : (a + b) * c = a * c + b * c := by sorry

example {a b : ℝ} : a / b = a * b⁻¹ := by sorry

example {a b c : ℝ} : a | b - c → (a | b ↔ a | c) := by sorry

example (s t : Set α) (x : α) : x ∈ s → x ∈ s ∪ t := by sorry

example (s t : Set α) (x : α) : x ∈ s ∪ t → x ∈ s ∨ x ∈ t := by sorry

example {a : α} {p : α → Prop} : a ∈ {x | p x} ↔ p a := by sorry

example {x a : α} {s : Set α} : x ∈ insert a s → x = a ∨ x ∈ s := by sorry

example {x : α} {a b : Set α} : x ∈ a ∩ b → x ∈ a := by sorry

example {a b : ℝ} : a ≤ b ↔ a < b ∨ a = b := by sorry

example {a b : ℤ} : a ≤ b - 1 ↔ a < b := by sorry

example {a b c : ℝ} (bc : a + b ≤ a + c) : b ≤ c := by sorry
```

请根据以下命名猜测并陈述出定理

```
/-
1. mul_add
2. add_sub_right_comm
3. le_of_lt_of_le
4. add_le_add
5. mem_union_of_mem_right
-/
```

6.5.2 tactics

```
variable (a b c d : ℝ)

#check pow_eq_zero
#check pow_two_nonneg
example {x y : ℝ} (h : x ^ 2 + y ^ 2 = 0) : x = 0 := by sorry
```

```

theorem aux : min a b + c ≤ min (a + c) (b + c) := by sorry

-- 你可以尝试使用aux来完成这一证明
#check le_antisymm
#check add_le_add_right
#check sub_eq_add_neg
example : min a b + c = min (a + c) (b + c) := by sorry

#check sq_nonneg
theorem fact1 : a * b * 2 ≤ a ^ 2 + b ^ 2 := by sorry

#check pow_two_nonneg
theorem fact2 : -(a * b) * 2 ≤ a ^ 2 + b ^ 2 := by sorry

-- 你可以使用上面两个定理来完成这一证明
#check le_div_iff
example : |a * b| ≤ (a ^ 2 + b ^ 2) / 2 := by sorry

-- Finish the proof using the theorems abs_mul, mul_le_mul, abs_nonneg, mul_lt_mul_right, and one_mul.
theorem my_lemma4 :
  ∀ {x y ε : ℝ}, 0 < ε → ε ≤ 1 → |x| < ε → |y| < ε → |x * y| < ε := by
  intro x y ε epos ele1 xlt ylt
  calc
    |x * y| = |x| * |y| := sorry
    _ ≤ |x| * ε := sorry
    _ < 1 * ε := sorry
    _ = ε := sorry

-- 使用calc
example (a b : ℝ) : - 2 * a * b ≤ a ^ 2 + b ^ 2 := by sorry

example (a b c : ℝ) : a * b + a * c + b * c ≤ a * a + b * b + c * c := by sorry

example {x y ε : ℝ} (epos : 0 < ε) (ele1 : ε ≤ 1) (xlt : |x| < ε) (ylt : |y| < ε) : |x * y| < ε := by sorry

def FnUb (f : ℝ → ℝ) (a : ℝ) : Prop :=
  ∀ x, f x ≤ a

def FnLb (f : ℝ → ℝ) (a : ℝ) : Prop :=
  ∀ x, a ≤ f x

section
variable (f g : ℝ → ℝ) (a b : ℝ)

example (hfa : FnLb f a) (hgb : FnLb g b) : FnLb (fun x ↦ f x + g x) (a + b) := by

```

```
    sorry

example (hfa : FnUb f a) (hgb : FnUb g b) (nng : FnLb g 0) (nna : 0 ≤ a) :
  FnUb (fun x ↦ f x * g x) (a * b) := by
    sorry

end
```

Chapter 7

Dot Notation

7.1 命名空间与点方法

本节介绍 Lean 里“点号.”的两种截然不同但外观相似的用途。阅读代码时，我们需要根据上下文迅速判断它是指层级结构还是函数调用：

- **命名空间前缀 (Namespace Prefix)**: 表示层级归属。例如 `Nat.add` 表示 `add` 定义在 `Nat` 命名空间下。
- **点方法 (Field/Dot Notation)**: 一种函数调用的语法糖。例如 `x.bar` 实际上是 `Foo.bar x` 的简写。

7.1.1 命名空间 (Namespaces)

Day2 简要介绍了命名空间用于组织代码，防止命名冲突。除了基本的 `namespace` 和 `open` 指令外，还有几个实用的细节。

1) 嵌套与简写

命名空间可以像文件夹一样嵌套。要在特定的命名空间内定义函数，我们既可以使用 `namespace` 代码块，也可以直接在定义名中使用点号。

```
namespace ns1

-- 1. 标准写法：在命名空间块内定义
namespace ns2
  def funcA : Nat → Nat := fun n => n
end ns2

-- 2. 语法糖：直接使用点号前缀
-- 这等同于手动打开 namespace ns2 ... end ns2
def ns2.funcB : Nat → Nat := fun n => n + 1

end ns1

-- 由于 funcA, funcB 被嵌套在两层 namespace 里，访问时需写全完整路径
#check ns1.ns2.funcA
#check ns1.ns2.funcB
```

2) open 的作用域

使用 open 指令可以省略前缀，但要注意它受当前 section 或 namespace 的作用域限制。

```
section
  open ns1.ns2
  #check funcA -- OK: 在 section 内可以直接访问
end

-- #check funcA -- Error: 出了 section, open 失效, 必须写全名
```

3) scoped notation 与 open scoped

这是 Mathlib 中非常常见的模式。普通的定义（如函数）通常是局部的，但记号（Notation）默认是全局的。

```
namespace Physics
def energy (m c : Nat) := m * c * c

notation "E" => energy
end Physics

#check E 10 1 -- Physics.energy 10 1 : Nat

namespace Logic -- 假设我又想在逻辑学里定义 "E" 表示 “存在 (Exists)”
def exists_prop := True

notation "E" => exists_prop

#check E -- 报错, ambiguous, possible interpretations

end Logic
```

为了防止记号污染全局环境（例如不想让 ∞ 到处都是），我们使用 scoped notation。

关键区别：

- open MySpace: 打开该空间下的所有普通定义（定理、函数），但不打开 scoped 记号。
- open scoped MySpace: 专门用于打开该空间下定义的 scoped 记号。

```
namespace MyMath
-- 定义一个 scoped 记号, 只有显式开启时才生效
scoped notation "∞" => (0 : Nat)

def myFunc (x : Nat) := x
end MyMath

-- #check ∞ -- Error: 记号未开启

section
-- 只打开记号, 不打开 namespace 里的函数
```

```
open scoped MyMath

#check ∞      -- 0 : Nat
-- #check myFunc -- Error: myFunc 依然需要前缀，因为没写 `open MyMath`
end
```

7.1.2 点方法 (Field Notation)

这是 Lean 中提升代码可读性的重要语法糖。如果一个命名空间名为类型名，则对于该命名空间下的定理或定义，若其需要一个类型为该命名空间名的参数，我们可以直接写该参数调用相关定理，而不是在前面加上命名空间前缀。这使得代码更接近面向对象语言的写法。

其核心规则是：

如果 $x : T$ ，那么 $x.f$ 会被解析为 $T.f \ x$

1) 基本例子

当 Lean 看到 $x.bar$ 时，它会检查 x 的类型（假设是 Foo ），然后查找 $Foo.bar$ 并将 x 作为第一个参数传入。

```
axiom Foo : Type

-- 命名空间名为类型名
def Foo.bar (_ : Foo) : String := "hello world!"

variable (a : Foo)

-- 以下两种写法完全等价：
#check Foo.bar a  -- 标准函数调用
#check a.bar      -- 点方法调用
```

第一行的 `axiom` 是用来声明公理的，所谓公理，就是无法证明对错的假设，这里相当于不管不顾、强行声明有这样一个类型 $Foo : Type$ 存在，为了下文的演示¹。

2) 参数穿透

点方法不仅限于第一个参数。如果函数定义中有多个参数，Lean 会尝试把点号前的变量匹配给类型符合的位置，其余参数照常写在后面。

```
variable (f : Foo)

-- 参数在前
def Foo.add (self : Foo) (_ : Nat) := self
#check f.add 42      -- 等价于 Foo.add f 42

-- 类型的参数不在第一位
-- bar 定义为：先接收 Nat，再接收 Foo
def Foo.bar₁ (_ : Nat) (self : Foo) := self
```

¹在正式的数学证明（如 Mathlib 开发）中，极度不推荐随意使用 `axiom`。因为如果你写了 `axiom H : 1 = 0`，虽然 Lean 会接受它（因为是公理），但这会导致整个逻辑系统崩溃

```
-- Lean 能识别出 f 是 Foo 类型应该填入第二个位置
#check f.bar₁ 10      -- 等价于 Foo.bar₁ 10 f
```

3) 管道符与链式调用

在处理多层嵌套时，可以使用管道符配合点方法。`|>` 是管道运算符，`x |>.f` 等价于 `(x).f`

例如：计算 0-99 之间偶数的平方，丢弃前 5 个，取接下来的 3 个求和。

- 普通写法（逻辑反向，括号地狱）：

```
-- 必须从最里层的 range 100 开始读，视线由于括号不断跳跃
#eval List.sum (List.take 3 (List.drop 5 (List.map (fun x => x^2) (List.filter (fun x => x % 2 == 0) (List.range 100)))))
```

- 点管道写法（流式阅读）：

```
#eval List.range 100
|>.filter (fun x => x % 2 == 0) -- 1. 选偶数
|>.map (fun x => x^2)          -- 2. 平方
|>.drop 5                     -- 3. 丢弃前5个
|>.take 3                     -- 4. 取前3个
|>.sum                        -- 5. 求和
```

本节小结：

- 看到 `List.map` 这种形式，它是命名空间索引。
- 看到 `L.map f`（其中 `L : List`），它是点方法，等价于 `List.map f L`。
- 使用 `open scoped` 来管理那些特殊的数学符号，保持环境整洁。

Chapter 8

逻辑表达

8.1 Lean 中的逻辑表达

8.1.1 命题逻辑

我们已经知道一个命题 $P : \text{Prop}$ 在 Lean 中被视为一个类型：

- 如果命题 P 为真，意味着类型 P 非空，即存在一个元素 $h : P$ 。我们称 h 为命题 P 的**证明**。
- 如果命题 P 为假，意味着类型 P 为空，不存在任何元素。

命题之间可以通过逻辑运算符组合。Lean 支持以下各类运算符：

- 零元运算符 (Prop): True (真), False (假)
- 一元运算符 ($\text{Prop} \rightarrow \text{Prop}$): $\neg$ (否定)
- 二元运算符 ($\text{Prop} \rightarrow \text{Prop} \rightarrow \text{Prop}$): $\wedge$ (合取), $\vee$ (析取), $\rightarrow$ (蕴含), $\leftrightarrow$ (等价)

下面我们介绍这些运算符在 Lean 中的定义及其对应的证明构造与使用方法。

1. True (真)

True 表示逻辑上的恒真命题。在 Lean 中，它有一个名为 `trivial` 的平凡证明。

```
#check True      -- True : Prop
#check trivial   -- trivial : True
```

2. False (假)

False 表示逻辑上的恒假命题。在 Lean 中，它没有任何证明（即类型为**空**）。

根据经典逻辑中的**爆炸原理** (Ex Falso Quodlibet)，如果假命题成立，则可以推出任意命题。在 Lean 中，这体现为 `False.elim`：如果我们拥有一个 False 的证明 h （即导出了矛盾），就可以利用 h 构造出任意命题的证明。

```
#check False     -- False : Prop

example (h : False) : 1 = 2 := by
  exact False.elim h
```

3. $\rightarrow$ (蕴含)

在 Lean 的类型论中，蕴含关系本质上是一个函数类型。要证明 $P \rightarrow Q$ ，等价于构造一个函数：它接受一个 P 的证明作为输入，并返回一个 Q 的证明作为输出。反之，如果我们已知 $h : P \rightarrow Q$ 且拥有 $p : P$ ，通过函数应用 $h\ p$ 即可得到 Q 的证明。

```
variable (P Q : Prop)
#check P → Q      -- P → Q : Prop
example (p : P) (h : P → Q) : Q := h p
```

4. $\neg$ (否定)

命题 $\neg P$ 表示 P 的否定。在 Lean 中，否定并不是一个原语，而是被定义为“蕴含假”：

$$\neg P \equiv P \rightarrow \text{False}$$

这表示：如果你能提供 P 的证明，我就能推出矛盾 (False)。因此，证明 $\neg P$ 就是构造一个从 P 到 False 的函数。

```
#check Not      -- Prop → Prop

variable (P : Prop)
#check ¬P       -- ¬P : Prop
example : (¬P) = (P → False) := rfl

example (p : P) (h : ¬P) : False := h p
```

利用上一章介绍的点号记法，（对于蕴含关系 $h : P \rightarrow Q$ ，我们可以直接使用 $h.\text{mt}$ (Modus Tollens) 其本来面貌是 Function.mt ，来获取其逆否命题的证明。

```
variable (h : P → Q)
#check h.mt      -- Function.mt h : ¬Q → ¬P
#check Function.mt {a b : Prop} : (a → b) → ¬b → ¬a
```

5. $\wedge$ (合取)

$P \wedge Q$ 表示 P 与 Q 同时成立。

构造证明：要证明 $P \wedge Q$ ，我们需要同时提供 P 的证明和 Q 的证明。可以使用构造函数 And.intro ，或者更常用的尖括号语法 $\langle p, q \rangle$ 。

```
#check And      -- Prop → Prop → Prop

variable (P Q : Prop)
example (p : P) (q : Q) : (P ∧ Q) := by
  exact And.intro p q

-- 使用尖括号语法的等价写法
example (P Q : Prop) (p : P) (q : Q) : P ∧ Q := by
  exact ⟨p, q⟩
```

使用证明：如果我们已知 $h : P \wedge Q$ ，可以通过 `And.left` (或 `h.left, h.1`) 提取出 P 的证明，通过 `And.right` (或 `h.right, h.2`) 提取出 Q 的证明。

```
variable (h₁ : P ∧ Q)
#check h₁.left    -- h₁.left : P
#check h₁.right   -- h₁.right : Q
#check h₁.1       -- h₁.1 : P
#check h₁.2       -- h₁.2 : Q
```

6. $\vee$ (析取)

$P \vee Q$ 表示 P 成立或者 Q 成立¹。

构造证明：只需要证明其中一个命题成立即可。

- 使用 `Or.inl` (left) 通过 P 的证明来构造 $P \vee Q$;
- 使用 `Or.inr` (right) 通过 Q 的证明来构造 $P \vee Q$ 。

```
variable (P Q : Prop)
example (p : P) : P ∨ Q := by
  exact Or.inl p

example (q : Q) : P ∨ Q := by
  exact Or.inr q
```

使用证明：如果我们已知 $h : P \vee Q$ ，想要证明目标 R ，则必须进行分类讨论。在逻辑上，这意味着我们需要证明“如果 P 成立则 R 成立”以及“如果 Q 成立则 R 成立”。在 Lean 中，这一过程由 `Or.elim` 实现。

`Or.elim` 接收三个显式参数：1. 一个“或”命题 $h : P \vee Q$ (讨论的基础)；2. 从 P 推出的证明 $P \rightarrow R$ ；3. 从 Q 推出的证明 $Q \rightarrow R$ 。

```
#check Or.elim -- Or.elim {a b c : Prop} (h : a ∨ b) (left : a → c) (right : b → c) : c

example (P Q R : Prop) (h : P ∨ Q) (h₁ : P → R) (h₂ : Q → R) : R := by
  apply Or.elim h
  • exact h₁ -- 分支 1: 证明 P → R
  • exact h₂ -- 分支 2: 证明 Q → R
```

7. $\leftrightarrow$ (等价)

$P \leftrightarrow Q$ 表示 P 当且仅当 Q 。从逻辑上讲，它等价于 $(P \rightarrow Q) \wedge (Q \rightarrow P)$ 。

构造证明：需要同时提供“前推后”和“后推前”两个证明。可以使用 `Iff.intro hpq hqp` 或尖括号 `<hpq, hqp>`。

```
variable (P Q : Prop)
example (hpq : P → Q) (hqp : Q → P) : P ↔ Q := by
  exact Iff.intro hpq hqp
```

¹Lean 默认合取的优先级高于析取，比如 $p \wedge q \vee p \wedge r$ 其实是 $(p \wedge q) \vee (p \wedge r)$

```
-- 使用尖括号语法的等价写法
example (hpq : P → Q) (hqp : Q → P) : P ↔ Q := by
  exact ⟨hpq, hqp⟩
```

使用证明: 如果已知 $h : P \leftrightarrow Q$, 我们可以使用 $h.mp$ (Modus Ponens) 获取 $P \rightarrow Q$, 使用 $h.mpr$ (Modus Ponens Reverse) 获取 $Q \rightarrow P$ 。

```
variable (h2 : P ↔ Q)

#check h2.mp      -- h2.mp : P → Q
#check h2.mpr     -- h2.mpr : Q → P
#check h2.1       -- h2.mp : P → Q
#check h2.2       -- h2.mpr : Q → P
```

类似于合取, 这里的 $.mp$ 和 $.mpr$ 也是点号记法的应用, 对应定理 $Iff.mp$ 和 $Iff.mpr$ 。

从上述代码的检查结果可以看出, 所有这些逻辑运算符操作的结果仍然是 $Prop$ 类型, 这保证了命题系统的封闭性。

8.2 一些进阶 Tactics

8.2.1 suffices, assumption 与 refine

在这一节中，我们补充几个能让我们证明脚本更加简洁、灵活的 tactics。

逆向推理：suffices

suffices 顾名思义，它的含义是“为了证明当前目标，只需证明命题 h 成立即可”。使用它会将证明过程分为两步：

1. 说明充分性：首先说明“为什么有了假设 h 就能推出原目标”。
2. 证明新目标：然后转而去证明这个新的假设 h 本身。

其基本语法格式为：

suffices 名称 (h) : 新的结论 from 通过 h 证明原目标

```
example (P Q R : Prop) (h1 : P → R) (h2 : Q) (h3 : Q → P) : R := by
-- 我们的目标是 R。
-- 我们意识到：只要有了 P，配合已有的 h1 : P → R，就能得到 R。
suffices h : P from h1 h

-- 现在，原目标 R 已经被“解决”了（通过 from 部分）。
-- 剩下的新目标变成了证明 P。
exact h3 h2
```

suffices 与我们之前学过的 have 非常相似，两者的区别主要在于 ** 思维顺序 **：

- have 是前向推理（Forward）：先证明引理 h 成立，再利用 h 去证明最终目标。（“我有工具 h ，所以我能解决问题。”）
- suffices 是后向推理（Backward）：先宣称引理 h 足够解决问题，再去回头寻找 h 。（“只要有工具 h 就够了，现在我去造个 h 。”）

我们可以在不同的情况下，根据直觉选择用起来更顺手的一个。

自动搜索：assumption 与 rwa

assumption 是一个非常直观且省力的 tactic。它的作用是搜索当前上下文中的所有假设，如果发现其中有一个假设与当前的目标完全匹配，则自动完成证明。

最简单的例子如下：

```
example (a b : ℝ) (h : a = b) : a = b := by
assumption -- 上下文中已经有了 h : a = b，直接匹配成功
```

稍微复杂一点的例子，结合了构造证明和反证法：

```
example {x y : ℝ} (h0 : x ≤ y) (h1 : ¬y ≤ x) : x ≤ y ∧ x ≠ y := by
constructor
• assumption -- 第一个分支 x ≤ y，上下文中已有 h0，直接解决
```

```
-- 反证法:  $x \neq y$  定义为  $x = y \rightarrow \text{False}$ 
intro h --  $\vdash \text{False}$ 
• apply h1 --  $\vdash y \leq x$ 
  rw [h] --  $y \leq y$  by rfl
```

组合技: 与assumption相关的一个 tactic 是 rwa。它的名称来源于 Rewrite + Assumption。它的行为类似于 rw，但在执行完重写操作后，会自动尝试调用一次 assumption。这在“重写一步后目标恰好变成已知条件”的场景下非常有用。

```
example (n : ℕ) (h : Prime n) : Nat.Prime n := by
  -- Nat.prime_iff : Nat.Prime p ↔ Prime p
  -- 重写后目标变为 Prime n, 正好与假设 h 匹配, rwa 一步到位
  rwa [Nat.prime_iff]
```

精准控制: refine

refine 的作用类似于 apply, 但 refine 在使用定理时, 需要提供所有的参数, 而没有像 apply 一样的自动推断功能。

```
example (a b c : ℝ) : a + b + c = a + (b + c) := by
  refine add_assoc a b c
```

但它也可以配合匿名构造子 ⟨...⟩ 使用, 直接构建出目标的结构 (如“且”、“存在”的骨架), 而 apply 无法直接“应用”一个构造子。

```
example (P Q R : Prop) (h1 : P) (h2 : Q) (h3 : P → R) : Q ∧ R := by
  refine ⟨h2, h3 h1⟩
```

利用占位符延迟证明

我们还可以通过 refine 先写出证明的“形状”, 通过混合使用引理和占位符 ?_ 延迟证明, 把注意力集中在当前能解决的部分, 将复杂的逻辑留给后续步骤。

```
example (p q r t : Prop) (h1 : p → q) (h2 : q → r) (h3 : p) (h4 : t) : r ∧ t := by
  refine ⟨?_, h4⟩
  exact h2 (h1 h3)
```

嵌套构造

我们甚至可以在一个 refine 留下的坑里, 再次使用 refine 一层一层地剥洋葱。

```
example (p q r : Prop) (h : p ∧ (q ∧ r)) : (p ∧ q) ∧ r := by
  refine ⟨?_, ?_⟩
  -- 子目标 1: 证明 p ∧ q
  • refine ⟨?_, ?_⟩
    -- 子目标 1.1: 证明 p
    • exact h.left
    -- 子目标 1.2: 证明 q
    • exact h.right.left

  -- 子目标 2: 证明 r
  • exact h.right.right
```

命名占位符

比如当证明目标是 $\leftrightarrow$ 时，我们可以通过起名 `?yourname` 清晰地拆分“前推后”和“后推前”两个目标。

```
example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) := by
  refine {?forward, ?backward}
  -- 此时 Lean 生成了两个标记为 forward 和 backward 的子目标,我们可以自由选择先证明哪一个
  case forward =>
    intro h
    -- 将 h : p ∧ (q ∨ r) 拆解为 hp 和 (hq | hr)
    rcases h with ⟨hp, hq | hr⟩
    • left; exact ⟨hp, hq⟩
    • right; exact ⟨hp, hr⟩

  case backward =>
    intro h
    -- 将 h : (p ∧ q) ∨ (p ∧ r) 分情况拆解
    rcases h with ⟨hp, hq⟩ | ⟨hp, hr⟩
    • exact ⟨hp, Or.inl hq⟩
    • exact ⟨hp, Or.inr hr⟩
```

8.2.2 函数相关: funext, apply_fun 与 gcongr

本节介绍在处理函数相等性、函数应用以及不等式同余时的常用 tactic。

函数相等性: funext

在数学中,要证明两个函数 $f, g : A \rightarrow B$ 相等,我们通常诉诸于函数外延性公理 (Function Extensionality): 如果对于任意的 $x \in A$, 都有 $f(x) = g(x)$, 那么 $f = g$ 。

在 Lean 中, funext (function extensionality) 正是用来实现这一逻辑的。

```
def f' (n : ℕ) := n + 1
def g (n : ℕ) := 1 + n

example : f' = g := by
  funext x -- 目标变为: f x = g x
  rw [f', g]
  rw [Nat.add_comm]
```

如果我们直接使用 funext 而不提供变量名, Lean 会自动引入一个无法直接访问的匿名变量 (通常在 InfoView 中显示为带剑号的变量, 如 $x†$)。为了在后续证明中引用这个变量, 我们需要使用 rename 将其“重命名”为可用的标识符。

```
example (A B : Type) (f g : A → B) : f = g := by -- 目标: f = g
  funext -- 目标变为: f x† = g x†
  rename A => x -- 将类型为 A 的匿名变量重命名为 x
  sorry
```

rename 的语法格式为 `rename 类型名 => 新变量名`。它会在上下文中查找指定类型的匿名变量, 并将其赋予新的名字。

函数应用：apply_fun

apply_fun 用于在等式或不等式的两边同时应用一个函数。根据它是作用于**假设**（正向）还是**目标**（逆向），其背后的数学逻辑和对函数的要求完全不同。

1. 作用于假设 (at hypothesis)

这是**正向推理**：我们已知某种关系，想推导出函数作用后的新关系。

对于等式 ($x = y$)：这是总是合法的。因为函数的定义保证了“若 $x = y$ 则 $f(x) = f(y)$ ”。

```
example (A B : Type) (x y : A) (f : A → B) (h : x = y)
  : f x = f y := by
  apply_fun f at h    -- 假设 h 变成了 `f x = f y`
  exact h
```

2. 作用于目标 (at goal)

这是**逆向推理**：我们想通过证明变换后的结论来反推原目标。这通常需要函数具有更强的性质，以保证逻辑的可逆性。

- 对于“不等于” ($x \neq y$)：这是总是合法的。逻辑基础是逆否命题：“若 $f(x) \neq f(y)$ 则 $x \neq y$ ”。不需要额外条件。

```
example (A B : Type) (x y : A) (f : A → B) : x ≠ y := by
  apply_fun f    -- 目标变为 `f x ≠ f y`
  sorry
```

- 对于等式 ($x = y$)：如果我们想将目标转化为 $f(x) = f(y)$ ，则必须保证 f 是**单射** (Injective)。Lean 会自动生成证明单射性的子目标。

```
example (A B : Type) (x y : A) (f : A → B) : x = y := by
  apply_fun f
  • -- 子目标 1: f x = f y
    sorry
  • -- 子目标 2: Injective f
    sorry
```

处理不等式：gcongr

在处理不等式（如 $\leq$ 或 $<$ ）时，我们通常不使用 apply_fun，而是使用更现代、更强大的 gcongr (Generalized Congruence)。

当我们的目标形如 $f(a) \leq f(b)$ ，且我们希望将其简化为 $a \leq b$ 时，gcongr 会自动分析函数 f 的结构并应用相应的单调性引理（它是一个自动化工具，在库里检索有 @[gcongr] 标签索引）。

场景：剥离函数的“外壳”

```
example (a b : ℝ) (h : a ≤ b) : a + 1 ≤ b + 1 := by
  -- 目标是证明 a + 1 ≤ b + 1
  -- gcongr 自动识别出两边都加了 1，这保持了不等式方向
  gcongr
```

```
example (a b : ℝ) (h : a ≤ b) : Real.exp a ≤ Real.exp b := by
  -- gcongr 自动识别出 Real.exp 是单调增的
  gcongr
```

注：这种“逆向”思维在 Lean 中非常常见。我们不是把假设 $a \leq b$ 变成 $e^a \leq e^b$ （正向），而是把目标 $e^a \leq e^b$ 简化回 $a \leq b$ （逆向），从而与假设匹配。

8.2.3 逻辑命题相关的实用 Tactic

在处理逻辑命题时，我们经常需要对假设进行拆解、利用矛盾进行推导，或者进行分类讨论。Lean 提供了一系列专门的 Tactic 来简化这些操作。我们将它们分为三类：结构拆解、反证与矛盾、以及分类讨论。

1. 结构拆解：obtain 与 rintro

这两个 Tactic 的核心功能是从复杂的命题中提取信息，它们都支持类似模式匹配的语法。

1. obtain

obtain 用于从现有的假设中“提取”出具体的成分。通俗地讲，它相当于 have（定义中间结论）加上 rcases（拆解）的组合。

典型场景：当你有一个结构复杂的假设，你想把它拆成几个简单的小假设（还是前面 refine 的例子）：

```
example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) := by
  refine {?forward, ?backward}

  case forward =>
    intro h
    obtain ⟨hp, (hq | hr)⟩ := h -- rcases h with ⟨hp, hq | hr⟩
    • left; exact ⟨hp, hq⟩
    • right; exact ⟨hp, hr⟩

  case backward =>
    intro h
    obtain (⟨hp, hq⟩ | ⟨hp, hr⟩) := h -- rcases h with ⟨hp, hq⟩ | ⟨hp, hr⟩
    • exact ⟨hp, Or.inl hq⟩
    • exact ⟨hp, Or.inr hr⟩
```

2. rintro

rintro 是 intro 和 rcases 的合体。它在引入假设的同时直接进行模式匹配拆解。优势：

- intro 虽然支持简单的构造拆解（如 ⟨hp, hq⟩），但不支持直接处理“或”命题（ $\vee$ ）。
- rcases 只能作用于上下文中已有的假设。
- rintro 则是在引入假设的同时直接完成拆解，并且支持处理 $\vee$ （使用 (h1 h2)| 语法），相当于前两者的结合。

（还是这个例子，此时我们可以合并 intro 和 rcases）

```
example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) := by
  refine {?forward, ?backward}
  case forward =>
    -- 拆解外层的 ∧ 为 ⟨hp, ...⟩; 拆解内层的 ∨ 为 hq | hr, 并自动分两个分支
    rintro ⟨hp, hq | hr⟩
    • left; exact ⟨hp, hq⟩
    • right; exact ⟨hp, hr⟩
  case backward =>
    -- 因为顶层就是 ∨, 所以用 ( ... | ... ) 括起来; 同时把两边的 ∧ 也拆了
    rintro (⟨hp, hq⟩ | ⟨hp, hr⟩)
    • exact ⟨hp, Or.inl hq⟩
    • exact ⟨hp, Or.inr hr⟩
```

2. 反证与矛盾: exfalso, by_contra 与 contrapose

这一组 Tactic 处理的是否定 (Negation) 和假命题 (False), 是经典逻辑证明中的利器。

1. exfalso

源自拉丁语 **ex falso quodlibet** (爆炸原理)。当你的假设中已经出现矛盾 (或者你能轻易推出 False) 时, 使用 exfalso 将当前目标直接变为 False。这通常用于“不论目标是什么, 只要前提矛盾就能得证”的情况。

```
example (p : Prop) (h : p ∧ ¬p) : 1 = 2 := by
  exfalso -- 目标从 1 = 2 变为 False
  exact h.right h.left -- 利用 ¬p 和 p 构造 False
```

2. by_contra

对应经典的反证法 (Proof by Contradiction)。如果你想证明 P , by_contra h 会让你假设 $\neg P$ (命名为 h), 并将目标变为 False。

```
example (p q : Prop) (h : ¬q → ¬p) : p → q := by
  intro hp
  -- 我们想证 q。尝试反证法: 假设 ¬q。
  by_contra hnq
  -- 现在我们需要寻找矛盾
  have hnp : ¬p := h hnq
  exact hnp hp -- p 和 ¬p 矛盾
```

3. contrapose. 它是指定了矛盾来源的反证法。使用 contrapose hq 会将当前目标与假设 hq 互换并取反。即: 它将证明目标变为 $\neg q$, 同时将假设 hq 更新为 $\neg p$ (利用了逆否命题 $q \rightarrow p \iff \neg p \rightarrow \neg q$)。

```
example (p q r : Prop) (hq : q) (hr : r) : p := by
  -- hq : q; hr : r ⊢ p
  contrapose hq
```

```
-- hr : r; hq : ¬p ⊢ ¬q
-- 目标与假设 hq 发生了互换并取反
sorry
```

若加上叹号 (如 `by_contra!` 或 `contrapose!`), `Tactic` 会自动对新生成的假设执行 `push_neg`, 将否定符号推入命题内部, 通常能得到更易处理的形式。

```
example (p q : Prop) (h : p → q) : ¬q → ¬p := by
  contrapose!
  exact h
```

3. 排中律与分歧: `by_cases`

`by_cases` 是最基础的分类讨论工具。对于任意命题 P , 它利用排中律将证明分为 P 成立和 $\neg P$ 成立两个分支。

注: Lean 默认采用构造主义逻辑, 但在处理一般数学命题时, `by_cases` 会自动调用经典逻辑公理 (Classical Logic), 因此我们可以对任意命题使用排中律, 而不必担心其是否可计算。

```
example (p q : Prop) : (p → q) ∨ (q → p) := by
  -- 这是一个经典的非构造性证明, 必须讨论 p 是否成立
  by_cases h : p
  • -- 情况 1: 假设 p 成立 (h : p)
    right
    intro _ -- 忽略 q
    exact h
  • -- 情况 2: 假设 p 不成立 (h : ¬p)
    left
    intro hp
    exfalso -- 从 hp : p 和 h : ¬p 推出矛盾
    exact h hp
```

- 建议总是写成 `by_cases h : p` 的形式, 以便给假设命名。
- 典型应用: 边界讨论。在处理除法或分段函数时, 经常需要讨论变量是否为 0。

Chapter 9

归纳与递归

9.0.1 归纳类型

在 Lean 的类型系统中，我们已经熟悉了如 `Prop`、`Type` 等宇宙层级，以及函数类型 $\rightarrow$ 。那么，诸如自然数 `Nat`、布尔值 `Bool` 这样具体的基础数据类型是如何产生的呢？它们都属于归纳类型 (Inductive Types)。

归纳类型可以被直观地理解为“上帝造物法则”：定义一个类型，本质上就是在定义这个类型的所有可能值的构造方式。只要列出了所有的“构造子” (Constructor)，也就穷尽了这个类型的所有可能性。

本节我们由简入繁展示如何定义归纳类型。

1. 枚举类型：有限的集合

最简单的归纳类型是只包含有限个值的类型。在其他编程语言中，这通常被称为“枚举” (enum)。例如 `Bool` 只有 `true` 和 `false` 两个值。

我们以定义“三原色”为例，使用 `inductive` 关键字：

```
inductive Color where
| Red    : Color
| Yellow : Color
| Blue   : Color
-- deriving Repr
```

语法解析：

- `inductive`：声明这是一个归纳类型。
- `where`：引出构造子列表（此处可省略，但建议保留以增加可读性）。
- `|` (竖线)：用于分隔不同的构造子。
- `Red, Yellow, Blue`：这是三个构造子的名字。在这里，它们不接受任何参数，直接成为了 `Color` 类型的值。

2. 递归类型：从有限到无限

枚举类型只能处理有限的情况。而我们完全可以有更复杂的 `constructor`，只需将其改为函数类型。比如要表示无穷集合（如自然数），我们需要引入递归。如果一个构造子接受该类型本身作为参数，就能派生出无穷的值。

最经典的例子是自然数 `Nat` 的皮亚诺定义：

```
inductive Nat' where
| zero : Nat'      -- 基础情况: 0
| succ : Nat' → Nat' -- 归纳步骤: 后继
```

这里我们用 `Nat'` 以区别于标准库的 `Nat`。这个定义告诉我们：

1. `zero` 是一个 `Nat'`（对应数字 0）。
2. 如果你有一个 `Nat'`（假设为 n ），那么 `succ n` 也是一个 `Nat'`。

通过这种方式，所有的自然数都可以被构造出来：

- $0 \Rightarrow \text{Nat'.zero}$
- $1 \Rightarrow \text{Nat'.succ Nat'.zero}$
- $2 \Rightarrow \text{Nat'.succ (Nat'.succ Nat'.zero)}$
- ...

这种“不同的构造方式对应不同的值”的特性，是归纳类型的核心。

3. 携带数据：链表与参数化

构造子不仅可以递归，还可以携带其他类型的数据。让我们定义一个专门存储自然数的链表 `NatList`：

```
inductive NatList where
| nil : NatList      -- 空链表
| cons : Nat → NatList → NatList -- 头部存一个 Nat，尾部接续 NatList
```

这虽然有用，但不够通用。我们显然不希望为 `Bool`、`String` 都单独定义一种链表。Lean 允许我们使用类型参数 (Type Parameters) 来定义通用的归纳类型。

我们可以定义一个对任意类型 α 都通用的列表 `List'`：

```
inductive List' (α : Type) where
| nil : List' α
| cons : α → List' α → List' α
```

此时，`List'` 本身成为了一个函数 `Type → Type`。只有当我们填入具体的类型参数（如 `List' Nat`）时，它才成为一个具体的归纳类型。

另一个常见的参数化类型是 `Option` (选项类型)，它用于处理可能为空的值（类似其他语言中的 `null/nil` 处理机制）：

```
inductive Option' (α : Type) where
| none : Option' α      -- 表示“无值”
| some : α → Option' α -- 表示“有值”，并携带该值
```

4. 结构体：单构造子的特例

有些时候，我们需要把多个不同类型的数据打包在一起（逻辑上的“且”关系）。例如，一个学生信息包含姓名、年龄和成绩。

虽然我们可以用 `inductive` 定义一个只有一个构造子的类型来实现：

```
inductive Student where
| mk : String → Nat → Nat → Student
```

但这有一个明显的缺点：参数是位置相关的。我们无法直观地知道这两个 `Nat` 哪个是年龄，哪个是成绩。对于这种“单构造子”情形，Lean 提供了语法糖 `structure`。

```
structure Student where
  name : String
  age : Nat
  grade : Nat
  deriving Repr
```

使用 `structure` 后，我们可以清晰地构造实例并访问数据：

```
def s : Student := { name := "Alice", age := 2, grade := 1 }

#eval s.name -- 输出 "Alice"
```

- **字段访问：**Lean 会自动为每个字段生成投影函数。写法 `s.name` 本质上是函数调用 `Student.name s` 的语法糖。这避免了我们手写模式匹配来提取数据。
- **底层实现：**它依然是一个归纳类型。Lean 会自动生成一个默认名为 `mk` 的构造子（全称 `Student.mk`），其类型为 `String → Nat → Nat → Student`。

5. 命题与类型的统一：同构与截断

回顾“命题即类型” (Curry-Howard 同构)，归纳类型在数据构造与逻辑证明中扮演了相同的代数角色。我们可以通过下表直观地看到这种统一性与差异：

	Type (数据宇宙)	Prop (命题宇宙)
乘法结构 ($\times$)	Product / Structure	And ($\wedge$)
加法结构 (+)	Sum ($\oplus$)	Or ($\vee$)

我们以加法结构为例：

```
-- Sum: 两个类型的交并 (Disjoint Union)
inductive MySum (α β : Type) where
| inl : α → MySum α β
| inr : β → MySum α β

-- Or: 两个命题的析取 (Disjunction)
inductive MyOr (a b : Prop) where
| inl : a → MyOr a b
| inr : b → MyOr a b
```

1. 可以看到两者的构造子 (`inl/inr`) 完全一致，都对应集合论中的不交并 $A \sqcup B$ 。本质上，它们都是通过“注入” (Injection) 来构造对象。
2. 区别在于它们所处的宇宙不同，导致了信息的处理方式不同：

- **Sum (保留信息):** 在 `Type` 中, `inl x` 与 `inr y` 是严格区分的。我们可以通过模式匹配提取出 x 或 y , 这对程序计算至关重要。
- **Or (擦除信息):** 在 `Prop` 中, `Lean` 强制执行证明无关性。这相当于对不交并进行了命题截断 (Quotient)。无论证明来自左侧还是右侧, 其携带的信息都被“商”掉了, 仅保留“存在证明”这一事实。

9.0.2 模式匹配

现在, 我们讨论如何使用这些归纳类型。这涉及到函数式编程中核心的机制: **模式匹配** (Pattern Matching)。在 `Lean` 中, 模式匹配主要通过 `match` 关键字实现。值得注意的是, 这种逻辑在编程和证明中是统一的:

- **在函数定义中:** 我们使用 `match` 来根据数据的不同构造形式返回不同的值。
- **在定理证明中 (Tactic Mode):** 我们使用 `cases` 策略。它的底层逻辑与 `match` 完全一致, 都是对目标进行拆解和分类讨论。

我们对上一节中的例子逐一应用模式匹配, 以展示其用法。

1. 基础匹配与相等性判定

以仅有三个值的 `Color` 类型为例。如果我们想要定义一个函数, 以一个 `Color` 作为输入, 对不同的颜色输出不同的结果, 初学者可能会想到使用 `if...then...else`:

```
def color2num (c : Color) :=
  if c == Color.Red then 1 else
  if c == Color.Yellow then 2 else 3
```

如果你直接运行这段代码, `Lean` 会报错。这是因为在 `Lean` 中, 当你定义一个新的类型时, 它默认是没有任何能力的。它并不知道如何判断两个 `Color` 是否相等。为了让 `==` 生效, 我们需要在定义类型时派生 (derive) 相应的实例:

```
inductive Color where
  | Red    : Color
  | Yellow : Color
  | Blue   : Color
deriving BEq, DecidableEq
```

- `BEq` (Boolean Equality): 允许我们使用 `==` 运算符进行计算, 返回 `Bool`。
- `DecidableEq` (Decidable Equality): 允许我们判定命题层面的相等 `=`, 返回 `Decidable` 证据。

虽然 `if` 语句行得通, 但它写起来显得冗长且不够直观。使用 `match` 关键字, 我们甚至不需要上述的 `BEq` 实例, 也不需要进行相等性运算, 而是直接根据构造子 (Constructor) 的结构进行拆解:

```
def color2num (c : Color) : Nat :=
  match c with
  | .Red    => 1
  | .Yellow => 2
  | .Blue   => 3
```

语法细节:

- **点号前缀:** `.Red` 中的点号是 `Lean` 的语法糖, 它允许 `Lean` 根据上下文 (`c` 的类型) 自动推断命名空间, 省略了 `Color.Red` 中的前缀。
- **穷举性:** `match` 必须覆盖所有可能的情况。对于有限的枚举类型, 这很容易做到。

2. 通配符与简写

如果我们只关心部分情况，可以使用下划线 `_` 作为**通配符**，匹配“所有剩余的情况”：

```
def isRedOrBlue (c : Color) : Bool :=
  match c with
  | .Red   => true
  | .Blue  => true
  | _      => false -- 匹配 Yellow
```

Lean 还提供了一种更紧凑的写法，允许我们在定义函数时直接省略 `match` 关键字，直接在参数列表后进行匹配。这在多参数匹配时尤为方便：

```
-- 这是一个类型为 Color → Color → Bool 的函数
def isRedAndBlue : Color → Color → Bool
  | .Red, .Blue => true
  | _, _       => false
```

3. 提取构造子参数

当归纳类型的构造子携带数据时（如 `Nat` 的 `succ`，或者结构体的字段），模式匹配不仅能区分情况，还能**提取**其中的数据。

递归类型的解构：以自然数的前驱函数 `pred` 为例：

```
def pred : Nat → Nat
-- 情况 1: 如果输入是 0
| .zero   => .zero
-- 情况 2: 如果输入是“某个 k 的后继”
| .succ k => k

#eval pred 0
#eval pred 3
```

在这里，`.succ k` 模式成功匹配后，变量 `k` 就代表了被包裹在 `succ` 中的那个自然数。

结构体的解构：

对于我们之前用 `inductive` 定义的 `Student` 类型（非 `structure` 语法糖版），我们无法直接用点号访问字段。但通过模式匹配，我们可以手动提取数据：

```
-- Student 的定义回顾: | mk : String → Nat → Nat → Student

def Student.name : Student → String
  | .mk name _ _ => name -- 提取第一个参数，忽略后两个

def Student.age : Student → Nat
  | .mk _ age _ => age -- 提取第二个参数

def Student.grade : Student → Nat
  | .mk _ _ grade => grade -- 提取第三个参数

def s := Student.mk "Alice" 20 100
#eval s.name
```

这揭示了 `structure` 的本质：`structure` 只是归纳类型的一个特例（只有一个构造子），而 `.name` 等访问器本质上就是 Lean 自动生成的上述模式匹配函数。

4. 综合示例：数据与逻辑

最后，我们来看一些依赖归纳类型的进阶例子。观察它们是如何处理类型参数和递归结构的。

和类型 (**Sum**) 与列表 (**List**):

```
-- 这是一个处理 MySum Nat (List Nat) 的函数
-- 如果左边是数字则返回该数字，如果右边是列表则计算列表和
def sum (x : MySum Nat (List Nat)) : Nat :=
  match x with
  | .inl n => n
  | .inr l => l.sum

-- 获取列表的最后一个元素（如果存在）
-- 这里展示了递归匹配 (recursive matching)
def List'.end {α : Type} : List' α → Option' α
  | .nil          => .none
  | .cons x .nil  => .some x    -- 匹配只有一个元素的情况
  | .cons _ l     => l.end      -- 递归调用
```

逻辑命题 (**Or**):

既然 `MyOr` (即命题层面的 `v`) 也是归纳类型，我们同样可以用 `match` 来使用它。在逻辑上，这对应于析取消除 (Disjunction Elimination)。

```
-- 已知 A 或 B 成立，且 A 不成立，则 B 必然成立
def MyOr.resolve_left {a b : Prop} (h : MyOr a b) (ha : ¬a) : b :=
  match h with
  | .inl h_a => absurd h_a ha -- 情况1: h 是 A 的证明。与 ¬A 矛盾，推出任意结论
  | .inr h_b => h_b           -- 情况2: h 是 B 的证明。直接返回
```

这里的 `absurd` 是一个利用矛盾 (`False`) 推出任意结论的函数，这在处理“不可能发生的分支”时非常有用。通过这些例子，我们可以深刻体会到 Lean 中“证明即程序”的统一美感。

Chapter 10

集合与子类型

10.1 集合

初学者使用 Lean 时往往容易产生这样一个直觉：可以将类型论中的“类型”类比为集合论中的“集合”，将类型的元素（Term）类比为集合中的元素。这种直觉虽然在一定程度上是自然的，但在严谨的数学形式化中，二者有着本质的区别。

我们以自然数为例。在标准集合论中，表达式 $(1, 3) \in \mathbb{N}$ 是一个合法的命题（虽然它的真值为假）。这是因为在集合论中，我们可以讨论任何对象是否属于某个集合。然而，在类型论中，表达式 $(1, 3) : \mathbb{N}$ 不仅是错误的，甚至是不合法的语法（ill-formed）。类型检查是在编译期进行的，这意味着我们无法构造出“类型不匹配”的表达式。因此，Lean 中的“类型”缺乏集合论中那种“从全集中选取特定子集”的灵活性。

为了在 Lean 中进行标准的集合论推理，我们需要在类型之上显式地定义“集合”的概念。

10.1.1 集合的定义

为了给 Lean 中的集合下定义，我们不妨从最基本的**属于关系**（membership）出发进行反向推导。

假设我们要在类型 α 上定义一个集合 s 。对于任意元素 $a : \alpha$ ，表达式 $a \in s$ 在数学上是一个命题（可能是真，也可能是假）。根据 Lean 的类型规则，我们可以得出两点事实：

- 表达式 $a \in s$ 的类型必须是 `Prop`。
- 这里的 s 实际上扮演了一个“判定者”的角色：它接收一个元素 a ，并返回一个关于该元素的命题。

因此，从函数的角度看，集合 s 的行为完全等同于一个函数：

输入类型： $\alpha \longrightarrow$ 输出类型：`Prop`

这意味着，集合本质上就是类型 α 上的一个谓词（**Predicate**）。

- 如果函数 s 作用于 a 得到的命题 $s\ a$ 为真，我们说 a 在集合中；
- 如果命题 $s\ a$ 为假（或无法证明），则 a 不在集合中。

基于这种逻辑，Lean 直接将集合定义为这个函数类型本身。为了使代码更具数学语义，我们定义 `Set  $\alpha$` 作为 $\alpha \rightarrow \text{Prop}$ 的别名。

```
variable ( $\alpha$  : Type)
```

```

-- Set  $\alpha$  的定义本质上就是  $\alpha \rightarrow \text{Prop}$ 
#check Set  $\alpha$       -- Set  $\alpha$  : Type
#print Set         -- def Set.{u} : Type u  $\rightarrow$  Type u := fun  $\alpha$  =>  $\alpha \rightarrow \text{Prop}$ 

example : (Set  $\alpha$ ) = ( $\alpha \rightarrow \text{Prop}$ ) := rfl

variable (s : Set  $\alpha$ )
#check s           -- s : Set  $\alpha$ 

```

在上述代码中，`#print` 命令展示了 `Set` 的具体定义是 `fun  $\alpha$  =>  $\alpha \rightarrow \text{Prop}$` 。即：当我们写下 `s : Set  $\alpha$` 时，我们实际上是在定义一个函数 `s :  $\alpha \rightarrow \text{Prop}$` 。这个定义简洁地捕捉了集合论中“外延公理”的精神：一个集合完全由它所包含的元素（即满足性质的元素）所决定。

根据上述定义，给定一个类型 α ，我们如何定义一个包含该类型全部元素的“全集”？（答案将在后续给出）

集合的性质与运算

基于 `Set  $\alpha$  :=  $\alpha \rightarrow \text{Prop}$` 这一核心定义，我们可以自然地导出集合论中的常见概念。所有的集合运算本质上都是逻辑运算的体现。

1. 属于关系 (Membership)

对于元素 `a :  $\alpha$` 和集合 `s : Set  $\alpha$` ，属于关系 `a  $\in$  s` 定义为性质 `s` 作用于参数 `a`，即 `s a`。

```

variable ( $\alpha$  : Type) (a :  $\alpha$ ) (s : Set  $\alpha$ )

#check a  $\in$  s      -- a  $\in$  s : Prop

-- 属于关系仅仅是函数应用的语法糖
example : (a  $\in$  s) = (s a) := rfl

```

Remark: 如果你在编辑器中按住 `Ctrl` 点击 `$\in$` 符号，可能会跳转到 `Membership.mem` 等复杂的类定义。这是 Lean 为了支持多种结构的属于关系（如列表、数组等）所做的抽象，但在集合（`Set`）的语境下，它就是简单的函数应用 `s a`。

同理，不属于关系 `a  $\notin$  s` 定义为属于关系的逻辑否定：

```

example : (a  $\notin$  s) =  $\neg$ (a  $\in$  s) := rfl

```

2. 子集关系 (Subset Relation)

在数学上，如果集合 `s` 中的任意元素都属于集合 `t`，我们就称 `s` 是 `t` 的子集，记作 `s  $\subseteq$  t`。

在 Lean 中，我们希望定义同样的表达式 `s  $\subseteq$  t`。由于这也是一个关于集合的断言，其类型应为 `Prop`。结合子集的数学含义——“对于任意 `x`，若 `x  $\in$  s` 则 `x  $\in$  t`”，我们可以直接将其转化为 Lean 的全称命题形式。

```

variable (t : Set  $\alpha$ )

#check s  $\subseteq$  t    -- s  $\subseteq$  t : Prop

-- 子集的定义等价于全称命题
example : (s  $\subseteq$  t) = ( $\forall$  x, x  $\in$  s  $\rightarrow$  x  $\in$  t) := rfl

```

Remark: 关于双花括号

细心的读者可能注意到了，上述定义中全称量词的变量 x 被 λx 括了起来，而不是我们常见的花括号 $\{ \}$ 。这被称为**严格隐式参数** (Strict-implicit binder)。

为了理解它的作用，我们需要对比一下它与普通隐式参数的区别：

- **普通隐式参数** $\{x\}$ ：只要你提到了包含该参数的函数名，Lean 就会立刻尝试推断 x 是什么。如果推断不出，它就会生成一个元变量 (metavariable，通常显示为 $?m$) 占位。
- **严格隐式参数** λx ：Lean 会等到你提供了后续的显式参数 (在这里是 $x \in s$ 这一项) 之后，才去推断 x 。

在谓词逻辑中，使用 λx 的好处在于：当我们只引用子集关系的证明 $h : s \subseteq t$ 而不传入参数时，Lean 不会自作聪明地把 h 展开成带问号的 $?m \in s \rightarrow ?m \in t$ ，而是保持 h 原本整洁的样子。这使得 Infoview 中的显示更加清晰。那么，当我们拥有一个子集关系的证明 $h : s \subseteq t$ 时，该如何使用它呢？根据定义， $s \subseteq t$ 本质上是一个函数：它接收“元素 x ”和“ $x \in s$ 的证据”，返回“ $x \in t$ 的证据”。由于 x 是隐式参数 (无论是严格还是普通)，我们在使用时只需要提供后者。

```
-- 假设我们知道 s 是 t 的子集，且 x 在 s 中
example (h : s ⊆ t) (x : α) (mem_s : x ∈ s) : x ∈ t := by
  -- h 的类型是 ∀ λx, x ∈ s → x ∈ t
  -- 我们像调用函数一样调用 h，只需传入 mem_s，Lean 会自动推断出 x
  exact h mem_s
```

3. 交集 (Intersection)

我们定义两个集合 s 和 t 的交集 $s \cap t$ 为：一个元素属于该交集，当且仅当它同时属于 s 和 t 。

```
#check s ∩ t      -- s ∩ t : Set α

-- 交集的定义: x ∈ s ∧ x ∈ t
example : (s ∩ t) = (fun x => x ∈ s ∧ x ∈ t) := rfl
```

Lean 中，交集本质上就是两个谓词的合取 (Logical And, $\wedge$) 但这里交集运算从定义上来讲并不是可交换的。即 $\text{fun } x \Rightarrow x \in s \wedge x \in t$ 和 $\text{fun } x \Rightarrow x \in t \wedge x \in s$ 虽然逻辑等价，但参数顺序不同，因此不是定义上相等的。

```
-- rfl会失败，因为左右两边并不是 definitionally equal
example : (s ∩ t) = (fun x => x ∈ t ∧ x ∈ s) := by rfl
```

由于元素 a 与交集 $s \cap t$ 之间的属于关系 $a \in s \cap t$ ，展开后就是 $a \in s \wedge a \in t$ 。因此我们可以按照处理“与”命题 (And) 的方法来处理它。

```
-- 1. 属于关系展开为合取
example : (a ∈ s ∩ t) = (a ∈ s ∧ a ∈ t) := by rfl

-- 2. 从交集中提取信息 (拆解)
example (h : a ∈ s ∩ t) : a ∈ s := by
  rcases h with ⟨mem_s, mem_t⟩ -- 将 h 拆解为两个假设
  exact mem_s

-- 或者直接访问项
example (h : a ∈ s ∩ t) : a ∈ s := h.left
```

```
-- 3. 证明元素属于交集（构造）
example (h1 : a ∈ s) (h2 : a ∈ t) : a ∈ s ∩ t := by
  constructor
    • exact h1      -- 证明 a ∈ s
    • exact h2      -- 证明 a ∈ t
```

4. 并集 (Union)

并集 $s \cup t$ 由属于 s 或者属于 t 的元素组成。逻辑上，这对应于命题的 **析取** (Or , $\vee$)。

```
example : (s ∪ t) = (fun x => x ∈ s ∨ x ∈ t) := by rfl
```

与交集的情形类似，一个元素 $a : \alpha$ 与并集 $s \cup t$ 之间的属于关系实际上就是一个“或”命题 $a \in s \vee a \in t$ ，通常需要使用 `rcases` 进行分类讨论：

```
-- 从并集中推理（分类讨论）
example (r : Set α) (h : a ∈ s ∪ t) (h1 : s ⊆ r) (h2 : t ⊆ r) : a ∈ r := by
  rcases h with mem_s | mem_t
    • exact h1 mem_s -- 情况 1: a ∈ s
    • exact h2 mem_t -- 情况 2: a ∈ t

-- 构造并集成员
example (h : a ∈ s) : a ∈ s ∪ t := by
  left -- 选择左侧分支
  exact h
```

5. 差集 (Difference)

差集 $s \setminus t$ 包含属于 s 但不属于 t 的元素。逻辑上，这是“与”和“非”的组合。

```
example : (s \ t) = (fun x => x ∈ s ∧ x ∉ t) := rfl

example (h : a ∈ s \ t) : a ∉ t := h.right

example (h1 : a ∈ s) (h2 : a ∉ t) : a ∈ s \ t := ⟨h1, h2⟩
```

6. 补集 (Complement)

补集 s^c 包含所有不属于 s 的元素。逻辑上，这对应于命题的否定。

```
#check sc      -- sc : Set α
example : sc = (fun x => x ∉ s) := rfl
```

处理补集即处理否定命题，常涉及反证或通过否定引入法 (`exfalso`):

```
-- 矛盾推导
example (h1 : a ∈ s) (h2 : a ∈ sc) : 1 = 2 := by
  exfalso
  exact h2 h1

-- 利用子集关系推导补集性质
```

```
example (h : a ∉ t) (h2 : s ⊆ t) : a ∈ sᶜ := by
  intro mem_s      -- 假设 a ∈ s
  apply h           -- 目标转变为证明 a ∈ t
  exact h2 mem_s    -- 利用子集关系得证
```

全集与空集

现在我们来回答本节前面提出的问题：如何定义一个包含类型 α 中全部元素的集合？

回想集合的定义：类型 α 上的集合 s 本质上是一个判定函数 $s : \alpha \rightarrow \text{Prop}$ 。如果我们希望集合包含所有元素，意味着这个判定函数对于任何输入 $x : \alpha$ 都应当返回“真”。在 Lean 中，最简单的恒真命题是 `True`（其证明为 `trivial`）。因此，全集可以定义为 `fun _ => True`。

Lean 标准库中已经为我们定义好了这个概念，记作 `Set.univ` (Universal Set)。

```
variable {α : Type}
-- 定义全集
def SET : Set α := fun _ => True

-- 证明任意元素都属于该集合
example : ∀ x : α, x ∈ SET := by
  intro x
  exact trivial

-- Lean 内置的全集
#check Set.univ      -- Set.univ : Set α

example : ∀ x : α, x ∈ Set.univ := by
  intro x
  exact trivial
```

相对地，我们可以定义数学中的**空集**。不难想到，空集对应的性质应当是对所有元素都为“假”。Lean 使用符号 $(\emptyset : \text{Set } \alpha)$ 来表示空集，其定义等价于 `fun _ => False`。

```
def EMPTY : Set α := fun _ => False

-- Lean 内置的空集符号
#check (∅ : Set α)

-- 证明没有任何元素属于空集
example : ∀ x : α, x ∉ (∅ : Set α) := by
  intro x mem_empty
  -- mem_empty 的类型是 x ∈ ∅, 即 False
  exact mem_empty
```

集合描述法 (Set Builder Notation)

为了贴近数学的自然语言习惯，Lean 提供了一套强大的**集合描述法**记号。给定类型 α 和其上的性质 $p : \alpha \rightarrow \text{Prop}$ ，我们可以使用 $\{x \mid p\ x\}$ 来定义由满足性质 p 的元素构成的集合。

```
variable (p :  $\alpha \rightarrow \text{Prop}$ )
#check {x :  $\alpha$  | p x}      -- {x | p x} : Set  $\alpha$ 
```

在这个记号中，竖线左边代表集合中的元素变量，右边代表该元素需满足的条件。

此外，Lean 还支持更高级的语法糖，允许我们在竖线左侧写构造表达式。例如，如果我们想定义“所有偶数的集合”，标准的写法是 $\{x \mid \exists n, 2 * n = x\}$ 。但在 Lean 中，我们可以简写为 $\{2 * n \mid n\}$ 。

```
-- 标准写法：所有满足“存在 n 使得 x = 2n”的 x
#check {x |  $\exists n, 2 * n = x$ }

-- 语法糖写法：由 n 构造出的 2n 构成的集合
#check {2 * n | n}
```

注：第二种写法 $\{2 * n \mid n\}$ 在底层会被 Lean 自动展开为第一种带存在量词的形式。这种写法虽然简洁，但在某些复杂类型推断场景下可能会受限。

函数的像与原像

函数与集合之间的交互是数学中的常见操作。在 Lean 中，我们主要关注**像**（Image）和**原像**（Preimage）。

1. **像 (Image)** 对于函数 $f : \alpha \rightarrow \beta$ 和集合 $s : \text{Set } \alpha$ ， f 在 s 上的像记作 $f '' s$ 。它是一个 β 上的集合，包含所有形式为 $f a$ （其中 $a \in s$ ）的元素。从逻辑定义上看，这涉及到一个**存在量词**： $y \in f(s) \iff \exists x \in s, f(x) = y$ 。

```
variable ( $\beta : \text{Type}$ ) (f :  $\alpha \rightarrow \beta$ ) (s : Set  $\alpha$ )

#check f '' s      -- f '' s : Set  $\beta$ 

example (b :  $\beta$ ) : (b  $\in$  f '' s) = ( $\exists a \in s, f a = b$ ) := rfl
```

2. **原像 (Preimage)** 对于函数 $f : \alpha \rightarrow \beta$ 和集合 $t : \text{Set } \beta$ ， t 在 f 下的原像记作 $f^{-1} t$ 。它是一个 α 上的集合，包含所有映射后落在 t 中的元素。原像的定义通常比像更简单，因为它直接利用了函数的性质，不涉及存在量词： $x \in f^{-1}(t) \iff f(x) \in t$ 。

```
variable (t : Set  $\beta$ )

#check f^{-1} t      -- f^{-1} t : Set  $\alpha$ 

example (a :  $\alpha$ ) : (a  $\in$  f^{-1} t) = (f a  $\in$  t) := rfl
```

有界量词 (Bounded Quantifiers)

在数学中，我们经常写 $\forall x > 0, P(x)$ 或 $\exists x > A, P(x)$ 。在逻辑底层，这两者的处理方式是不同的：

- **全称量词与蕴含**： $\forall x \in s, p x$ 实际上是 $\forall x, x \in s \rightarrow p x$ 的缩写。即：“对于任意 x ，如果 x 在 s 中，那么 $p x$ 成立”。
- **存在量词与合取**： $\exists x \in s, p x$ 实际上是 $\exists x, x \in s \wedge p x$ 的缩写。即：“存在一个 x ，它既在 s 中，且满足 $p x$ ”。

这对应 Lean 中常用的一种语法糖：有界量词

```
variable (p :  $\mathbb{N} \rightarrow \text{Prop}$ )

-- 全称量词的限制条件转化为蕴含 ( $\rightarrow$ )
example : ( $\forall x > 0, p\ x$ ) = ( $\forall x, x > 0 \rightarrow p\ x$ ) := by rfl

-- 存在量词的限制条件转化为合取 ( $\wedge$ )
example : ( $\exists x > 0, p\ x$ ) = ( $\exists x, x > 0 \wedge p\ x$ ) := by rfl
```

10.2 子类型 (Subtype) 与类型转换

集合与类型的界限

按照我们最初介绍的类型论原则，类型 $\alpha : \text{Type}$ 上的一个集合 $s : \text{Set } \alpha$ 本身是一个项 (Term)，而不是一个类型 (Type)。从严格的语法角度来看，表达式 $a : s$ (意指 a 是类型 s 的一个实例) 应当是无意义的，因为 $:$ 右侧必须是一个类型。

然而，如果我们尝试在 Lean 中通过代码实现这一看似错误的操作，会发生什么呢？

```
variable ( $\alpha : \text{Type}$ ) ( $s : \text{Set } \alpha$ )

variable ( $a : s$ )    -- 竟然没有报错!!

#check a             --  $a : \uparrow s$ 
```

我们惊讶地发现，Lean 接受了 $a : s$ 这样的表达式。但请注意 #check 的输出结果：变量 a 的类型在 Infview 中被显示为 $\uparrow s$ ，而不是简单的 s 。这里的上箭头 $\uparrow$ 暗示了某种隐式的转换正在发生。

10.2.1 子类型的定义与结构

为了理解上述现象，我们需要引入子类型 (Subtype) 的概念。

对于一个类型 α 和其上的性质 $p : \alpha \rightarrow \text{Prop}$ ，我们可以从 α 中筛选出所有满足性质 p 的元素，构成一个新的类型。我们将其记作 $\{x : \alpha \mid p\ x\}$ ，称之为 α 的一个子类型。

现在回到集合的问题上。回顾定义，集合 $s : \text{Set } \alpha$ 本质上就是谓词 $s : \alpha \rightarrow \text{Prop}$ 。因此，Lean 通过一种被称为强制类型转换 (Coercion to Sort) 的机制，自动将集合 s 提升为了子类型 $\{x : \alpha \mid x \in s\}$ 。这也解释了为什么 $a : \uparrow s$ 是合法的——它实际上代表 $a : \{x : \alpha \mid x \in s\}$ 。

子类型的内部结构

一个子类型的元素 $a : \{x : \alpha \mid p\ x\}$ 实际上是一个包含两个分量的“包”：

1. 数值分量 $a.\text{val} : \alpha$ 。这是来自原类型 α 的具体元素。
2. 性质分量 $a.\text{property} : p\ a.\text{val}$ 。这是证明该元素满足性质 p 的证据 (Proof)。

Lean 为访问这两个分量提供了简便的记号 (语法糖)： $a.1$ 等价于 $a.\text{val}$ ， $a.2$ 等价于 $a.\text{property}$ 。

```
variable ( $\alpha : \text{Type}$ ) ( $p : \alpha \rightarrow \text{Prop}$ ) ( $a : \{x : \alpha \mid p\ x\}$ )

#check a.val          --  $\uparrow a : \alpha$ 
#check a.property     --  $a.\text{property} : p\ \uparrow a$ 
#check a.1            --  $\uparrow a : \alpha$ 
#check a.2            --  $a.\text{property} : p\ \uparrow a$ 

-- 底层定义
#check Subtype.val     -- ... (self : Subtype p) :  $\alpha$ 
#check Subtype.property -- ... (self : Subtype p) :  $p\ \uparrow \text{self}$ 
```

实战与陷阱：多层子类型

让我们通过实数集的例子来深入理解。假设我们想定义非负实数类型，我们可以使用性质 $\text{fun } r \Rightarrow r \geq 0$ ：

```
#check {x : ℝ // x ≥ 0}      -- { x // x ≥ 0 } : Type
variable (y : {x : ℝ // x ≥ 0})

#check y.val                -- ↑y : ℝ
#check y.property           -- y.property : ↑y ≥ 0
```

如果我们要更进一步，从这个“非负实数类型”中再取出“正实数”构成一个新的类型，会发生什么？这在定义上当然是可行的，但需要格外小心谓词的写法。

错误写法： $\{x : (\{x : \mathbb{R} // x \geq 0\}) // x > 0\}$ 这里的 x 本身是一个子类型（非负实数），我们不能直接拿它和 0 （实数）做比较。

正确写法： $\{x : (\{x : \mathbb{R} // x \geq 0\}) // x.val > 0\}$ 我们需要取出 x 所代表的实数值 $x.val$ 来与 0 进行比较。

```
-- 定义嵌套子类型：非负实数中的正数
#check {x : {x : ℝ // x ≥ 0} // x.val > 0} -- { x // ↑x > 0 } : Type
variable (z : {x : {x : ℝ // x ≥ 0} // x.val > 0})

-- 第一层解包：得到非负实数
#check z.val          -- ↑z : { x // x ≥ 0 }
-- 第一层性质：证明该非负实数的“值”大于 0
#check z.property     -- z.property : ↑↑z > 0

-- 第二层解包：得到原始实数
#check z.val.val      -- ↑↑z : ℝ
-- 第二层性质：证明该实数非负
#check z.val.property -- (↑z).property : ↑↑z ≥ 0
```

嵌套子类型的代价

虽然 Lean 允许通过 $z.val.val$ 这种方式层层剥洋葱，但在实践中，**不推荐**定义这种多层嵌套的子类型。层数越多，访问底层数据和性质就越繁琐（如 $x.val.val.val...$ ），这会给证明和代码维护带来巨大的认知负担。

特别是当我们在集合上定义集合时，很容易陷入这个陷阱：比如对于类型 α 来说，我们令 $s : \text{Set } \alpha$ 是 α 上的一个集合，那么 $t : \text{Set } s$ 相当于先把 s 视作 α 的子类型，然后再定义了这个子类型上的一个集合 t ，这时候，如果我们使用 $x : t$ 把 t 再变成 s 的子类型：

```
variable (α : Type) (s : Set α) (t : Set s) (x : t)

-- x 是 t 的元素，t 是 s 上的集合
-- 要想拿到 x 对应的 α 中的元素，我们需要剥两层皮
#check x.val.val      -- ↑↑x : α
#check x.val.property -- (↑x).property : ↑↑x ∈ s
```

实践建议：在使用 $\text{term} : \text{type}$ 这种表达式时，请务必审视 type 的本质。如果需要表达“满足性质 A 且满足性质 B 的元素”，通常更好的做法是直接 在原始类型 α 上定义一个组合性质 $\text{fun } x \Rightarrow A \ x \wedge B \ x$ ，从而构造一个单层的子类型，而不是通过嵌套构建。

10.2.2 类型转换 (Coercion)

现在我们来解释上一节中遇到的箭头符号 $\uparrow$ 到底是什么意思。

问题引入：数学直觉与类型严格性

在常规数学中，我们习惯于存在一个自然的包含链： $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$ 。这意味着一个自然数（如 1）在任何需要实数的场合，都可以被无条件地当作实数 1 来使用。然而，Lean 是建立在严格类型论基础上的。在 Lean 中， $1 : \mathbb{N}$ 和 $1 : \mathbb{R}$ 是完全不同的两个对象，它们的类型截然不同。

解决方案：隐式强制转换

为了在保持类型系统严格性的同时提供便利，Lean 引入了**强制类型转换**（Coercion）机制。

当 Lean 遇到类型不匹配的表达式时（例如需要 $\mathbb{R}$ 却得到了 $\mathbb{N}$ ），它并不会立即报错，而是会尝试在后台搜索是否存在一个已注册的“转换函数”，能将当前类型转化为目标类型。

如果找到了这样的函数，Lean 就会自动插入它。在 Infoview 的显示中，这种自动插入的转换函数通常被标记为一个向上的箭头 $\uparrow$ 。

```
-- 定义一个实数上的函数
def f' :  $\mathbb{R} \rightarrow \mathbb{R}$  := fun x => x + 1
-- 数学上  $f(1) = 2$  是直接的，但在 Lean 中，1 默认为自然数
#check f' (1 :  $\mathbb{N}$ )    -- f  $\uparrow$  1 :  $\mathbb{R}$ 
```

在上例中，Lean 自动插入了从 $\mathbb{N}$ 到 $\mathbb{R}$ 的转换函数， $\uparrow 1$ 实际上代表“应用了转换函数后的 1”。这个 $\uparrow$ 并不是一个具体的函数名，而是发生了一次类型转换的视觉标识。

在 Lean 的 Infoview 中，根据类型转换的目标不同，我们会看到三种不同形状的箭头。它们均表示发生了类型转换，但具体含义有所不同。

1. $\uparrow$. 这个箭头符号没有更多特殊含义，它只表示在箭头处发生了一个类型转换，至于转换成了什么类型，我们无法从记号上直接看出。值得注意的是， $\uparrow x + \uparrow y$ 表达式和 $\uparrow(x + y)$ 表示的含义不同，前者表示先将 x 和 y 转到某个目标类型后，再做加法，而后者表示现在原类型上将 x 和 y 相加，然后把相加的结果转到目标类型。
2. $\downarrow$. 这个箭头符号也表示发生了类型转换，并且类型转换的目标类型是 `Type`，即把原表达式变成了一个类型。
3. $\hookrightarrow$. 这个箭头符号也表示发生了类型转换，并且类型转换的目标类型是一个函数类型。

实践建议：显式标注

虽然隐式转换非常方便，但在复杂的数学表达式中，过度依赖 $\uparrow$ 可能会导致 Lean 推断出的目标类型不符合你的预期，或者让表达式难以阅读。

在实际工作中，如果类型转换的目标不明显，我们建议使用**显式类型标注**（Type Ascription）来指明意图。

```
-- 显式指明我们想要有理数
#check (1 :  $\mathbb{Q}$ )

-- 显式指明我们想要实数
#check (1 :  $\mathbb{R}$ )
```

10.2.3 类型转换处理 tactic

在处理涉及自然数、整数、有理数等混合运算时，类型转换（Coercion, $\uparrow$ ）经常会让证明变得繁琐。Lean 提供了两个专门的 tactic 来处理这种情况，它们的行为恰好相反，分别对应着“化简”与“展开”两种策略。

`norm_cast` **向外提**。它会尝试把所有的类型转换符号 $\uparrow$ 移动到表达式的最外层。例如，将 $\uparrow x + \uparrow y$ 转化为 $\uparrow(x + y)$ 。通常情况下，这是我们首选的策略。因为它倾向于消除表达式内部类型转换，将问题还原为原始类型（如 $\mathbb{Z}$ 或 $\mathbb{N}$ ）上的纯粹运算，从而简化证明。

`push_cast` **向内推**。它会尝试把类型转换符号 $\uparrow$ 推入到表达式的内部，作用于每一个子项。例如，将 $\uparrow(x + y)$ 转化为 $\uparrow x + \uparrow y$ 。虽然这看起来会让表达式变复杂（增加了 $\uparrow$ 的数量），但在某些特定场景下——例如当我们需要在目标类型（如 $\mathbb{Q}$ ）上应用某些特定的引理，或者需要对齐等式两边的结构时——它是必不可少的。

```
-- norm_cast 示例：消除类型转换，回到整数加法
example (a b :  $\mathbb{Z}$ ) (h : (a :  $\mathbb{Q}$ ) + b < 10) : a + b < 10 := by
  norm_cast at h

example (a b :  $\mathbb{Z}$ ) (h : (a :  $\mathbb{Q}$ ) + (b :  $\mathbb{Q}$ ) = 1) : ((a + b :  $\mathbb{Z}$ ) :  $\mathbb{Q}$ ) = 1 := by
  --  $\vdash \uparrow(a + b) = 1$ 
  push_cast
  --  $\vdash \uparrow a + \uparrow b = 1$ 
  exact h
```

从上面的例子可以看到，这两个 tactic 不仅可以作用于目标（Goal），也可以通过 `at h` 作用于假设（Hypothesis），非常灵活。

10.3 更多自动化 Tactic

本节将介绍几个通用的自动化工具。它们不局限于集合论，在处理涉及数值、代数运算的问题时也非常常用。

10.3.1 外延性 (ext)

在数学中，许多对象是由其“成分”唯一确定的。例如：

- 两个集合相等，当且仅当它们包含相同的元素。
- 两个函数相等，当且仅当它们对所有输入都有相同的输出。
- 两个有序对相等，当且仅当它们的每一个分量都相等。

`ext` 用于将对象的相等性证明转化为其成分的相等性证明。Lean 会自动在库中查找标记为 `@[ext]` 的外延性引理并应用它。

```
variable (α β : Type)

-- 1. 用于集合：转化为元素互推
example (s t : Set α) (h : ∀ x, x ∈ s ↔ x ∈ t) : s = t := by
  ext x
  exact h x

-- 2. 用于函数：转化为函数值相等
example (f g : α → β) (h : ∀ x, f x = g x) : f = g := by
  ext x
  exact h x

-- 3. 用于结构体（如子群）：转化为底层集合相等（子群相等本质上就是它们包含的元素相等）
example {G : Type*} [Group G] (H K : Subgroup G) (h : ∀ x, x ∈ H ↔ x ∈ K) : H = K := by
  ext x
  exact h x
```

注：

- 名字 `x` 是可选的：可以定义成其他名字，也可以不加名字，后者仍然会自动生成一个被 `†` 标记的不可访问的变量名。
- 与 `funext` 的区别：`funext` 是专门用于证明函数相等性的策略（Function Extensionality）。相比之下，`ext` 是一个更强大的通用工具。适用于函数、集合、结构体等所有注册了 `@[ext]` 的类型。另外 `ext` 通常会递归地应用，一次性剥离多层结构，而 `funext` 通常只剥离一层参数。

10.3.2 数值判定与代数化简

这一类 Tactic 主要用于处理具体的数值符号判定和代数式的结构化简。

`positivity` 用于自动化证明形如 $0 \leq x$, $0 < x$ 和 $x \neq 0$ 的情形。这是一个结束性 tactic，因此，它要么解决了目标，要么报告一个错误。其作用原理有些复杂，通俗地讲，`positivity` 会对复杂表达式结合变量数值下界进行解析，拆分成一个个简单的以上三种情形的数值表达式，然后分别进行证明。

```
example {a b : ℝ} (h : b ≠ 0) : 0 < a^2 + |b| + 1 := by
  positivity
```

面对上述目标，positivity 会执行以下步骤：

1. 识别出顶层运算是加法 ($x + y + z$)。要证明和为正，策略试图证明每一项均非负，且至少有一项严格为正。
2. 识别出这是平方形式。根据内置规则（平方数总是非负），判定 $a^2 \geq 0$ 。
3. 识别出这是绝对值形式。虽然绝对值天然只保证 ≥ 0 ，但策略扫描了局部上下文，发现了假设 $h : b \neq 0$ 。结合绝对值的性质（非零数的绝对值严格为正），判定 $|b| > 0$ 。
4. 识别出这是数字字面量。策略内部调用 norm_num 进行计算，判定 $1 > 0$ 。
5. 将结果合并， $(\geq 0) + (> 0) + (> 0)$ 显然是严格大于 0 的。

positivity 的能力依赖于表达式的结构。对于像 $0 < 5 - 2$ 这种涉及减法的表达式，由于“正数减正数”符号不确定，它通常无法直接处理（除非配合 norm_num 先进行化简）。

```
example (x : ℝ) (hx : 0 < x) : 0 < 5 - (-x) := by
  -- positivity -- 失败！因为它看不出 5 - (-x) 是正的
  norm_num      -- 化简为 0 < 5 + x
  positivity    -- 成功
```

field_simp 用于化简包含除法（逆元）的表达式。它通常会将分式通分，消除分母。常与 ring（处理环上的加减乘）配合使用来解决四则运算问题。

```
example (x : ℝ) (h : x > 0) : 1 / x + 1 = (x + 1) / x := by
  field_simp
  ring

-- 可以通过参数列表传入自定义引理
example {a b : ℝ} (h : b ≠ 1) : a = (a * b - a) / (b - 1) := by
  field_simp [sub_ne_zero_of_ne h] -- 告诉 tactic 分母不为 0
  ring
```

10.3.3 综合性求解器

这一类 Tactic 功能更为强大，通常用于直接解决某一类特定领域的复杂命题。

算术自动化： omega 是 Lean 4 中功能极为强大的自动化 tactic，用于处理自然数（Nat）和整数（Int）的一切命题。

它的底层机制基于 Presburger 算术的决策过程。通俗来讲，只要你的命题只涉及加法、减法、乘常数（如 $2 * x$ ）以及比较关系（ $\leq, <, =$ ），omega 几乎都能秒杀。

omega 的能力目前对于除法和取模有所限制：

- **支持：**自然数的除法（下取整）。
- **受限：**取模运算（%）。由于取模引入了非线性特性，omega 目前只能处理非常简单的取模情形。

尽管如此，随着 Lean 的迭代，omega 的功能正在不断完善，它在未来可能会成为处理整数相关命题的神器。

```
-- 线性不等式：轻松解决
example (x : Nat) : x ≥ 1 → x + 1 ≤ 2 * x := by omega

-- 包含除法：可以解决
example (x : Int) : x ≥ 2 → x / 2 ≤ x - 1 := by omega

-- 简单的数值取模：可以解决
example : 5 % 2 = 1 := by omega

-- 涉及定义的取模：需要辅助
example (x : ℕ) (h : Odd x) : x % 2 ≠ 0 := by
  rw [Odd] at h -- 先展开 Odd 的定义
  omega        -- 然后交给 omega

-- 复杂的取模：需要提供定理
example (x : Nat) : x * (x + 1) % 2 = 0 := by
  have h : Even (x * (x + 1)) := by
    exact Nat.even_mul_succ_self x
  rw [Even] at h -- 利用偶数定义将取模转化为存在性命题
  omega
```

可以看到，在最后两个例子中，面对 omega 无法直接“看穿”的高级概念（如 Odd、Even），我们需要手动展开定义或引入辅助引理，将问题转化为 omega 能理解的算术形式。

通用证明搜索： aesop (Automated Extensible Search for Obvious Proofs) 是 Lean 4 中一个通用的自动化证明搜索 tactic，旨在替代 Coq 中的 auto 或 Isabelle 中的 auto/blast。

与 omega 不同，aesop 的底层机制是一种“最优优先的树搜索”（Best-first Tree Search）。在搜索过程中，它会自动尝试应用各种规则，并将这些规则分为两类：

- 安全规则 (Safe Rule)：应用后肯定不会导致证明“走入死胡同”的操作（如 intro, split）。**策略：** aesop 会贪婪地应用所有可用的安全规则。这相当于“无脑推进”，不需要回溯。
- 非安全规则 (Unsafe Rule)：应用后可能会导致证明无法完成的操作（如尝试 apply 某个特定的定理，或者进行某个特定的分类讨论）。**策略：** 这是搜索树产生分支的地方。aesop 会根据启发式评分，优先尝试可能性最大的分支。如果走不通，它会自动回溯并尝试其他路径。

此外，aesop 的另一个强大之处在于它内置了 simp。在搜索树的每一个节点，它都会先尝试用 simp 进行标准化和化简。因此，它实际上是“逻辑拆解 (Safe/Unsafe)”与“代数化简 (Simp)”的结合体，非常适合消灭那些“逻辑琐碎但步骤繁多”的证明

```
-- 已知：存在一个 x，使得 P x 且 Q x；并且：对所有 x，P x → R x；证明：存在一个 x，使得 R x ∧ Q x
example (α : Type*) (P Q R : α → Prop) (h₁ : ∃ x, P x ∧ Q x) (h₂ : ∀ x, P x → R x) : ∃ x, R x ∧ Q x := by
  aesop -- 一阶逻辑 + 构造性目标

open Topology
example {α β : Type*} [TopologicalSpace α] [TopologicalSpace β]
  (f g : α → β) (hf : Continuous f) (hg : Continuous g) :
```

```
Continuous fun x => (f x, g x) := by
aesop -- aesop 自动把目标匹配到库里现成引理然后 apply 掉
```

值得一提的是，如果你想知道 `aesop` 到底干了什么，可以使用 `aesop?`。它会在 `Infoview` 中打印出搜索到的具体证明步骤（Try this: ...），方便你学习或将其替换为显式的 `Tactic` 脚本。

Chapter 11

群

11.1 Mathlib 中的群结构

在 Mathlib 中，群论的构建并非从零开始，而是建立在幺半群（Monoid）的层级之上。这种设计主要是为了兼容环（Ring）的乘法结构，因为环的乘法通常只构成幺半群而非群。

从 Monoid 到 Group

Mathlib 通过类型类（Type Class）定义代数结构。Group G 本质上是一个拥有逆元（Inv）的 Monoid G 。这意味着我们在群论中使用的许多引理实际上继承自 Monoid 库。

- Monoid M : 满足结合律且有单位元的结构（通常用于环的乘法或自然数加法）。
- Group G : 在 Monoid 基础上增加了 div 或 inv 结构，即每个元素均可逆。

我们可以先通过以下等效代码来理解 Lean 中群的核心定义：

```
namespace Demo
class Group (G : Type*) extends Mul G, One G, Inv G :=
  (mul_assoc : ∀ a b c : G, (a * b) * c = a * (b * c))
  (one_mul : ∀ a : G, 1 * a = a)
  (mul_one : ∀ a : G, a * 1 = a)
  (mul_left_inv : ∀ a : G, a-1 * a = 1)
end Demo
```

这里 extends 的意思是，要成为一个 Group，类型 G 必须先拥有乘法（Mul）、单位元（One）和求逆（Inv）这三种基本运算结构。

符号系统与对偶性

Mathlib 严格区分乘法记号（Multiplicative，默认）和加法记号（Additive）。这两套系统通过元程序 to_additive 自动关联，但在代码层面是两个独立的类型类实例。

```
-- 整数加法构成一个交换加法群
#check AddCommGroup ℤ
-- 非零实数乘法构成一个交换群
#check CommGroup ℝ*
```

11.2 群同态、子群与商群

本节我们将深入探讨群论中的结构性概念：群同态、子群以及商群。Lean 通过精巧的类型设计（如打包结构、强制转换和类型类）来处理这些概念。

11.2.1 群同态 (Group Homomorphisms)

在介绍群同态之前，我们需要先理解 Mathlib 中处理代数同态的核心设计理念。

打包映射 (Bundled Maps)

在纸笔数学中，我们通常说“令 f 为一个从 G 到 H 的同态”。但在 Lean 中，为了计算机能自动管理这些性质，同态被定义为**打包结构** (Bundled Structure)。

这意味着一个对象 $f : G \rightarrow H$ 不仅仅是一个函数，它是一个包含两部分信息的**结构体**：

1. 底层的映射函数 (Map)。
2. 该映射保持运算性质的证明 (Properties)。

我们使用 `MonoidHom M N` (记作 $M \rightarrow^* N$) 来表示乘法同态，用 `AddMonoidHom M N` (记作 $M \rightarrow_+ N$) 表示加法同态。

```
variable {M N : Type*} [Monoid M] [Monoid N]

-- 使用点记法 (Dot Notation) 调用性质
-- f.map_mul x y 等价于 MonoidHom.map_mul f x y
example (x y : M) (f : M →* N) : f (x * y) = f x * f y := f.map_mul x y

variable {A B : Type*} [AddMonoid A] [AddMonoid B]

-- 加法版本：保持零元
example (f : A →+ B) : f 0 = 0 := f.map_zero
```

群同态的组合

由于同态是打包结构，我们**不能**直接使用普通的函数组合符号 $\circ$ ，因为 $g \circ f$ 的类型仅仅是一个普通函数，它丢失了“保持代数结构”的信息。

因此，Mathlib 提供了专用的组合函数 `MonoidHom.comp`。

```
variable {P : Type*} [Monoid P]

-- g.comp f 意味着先应用 f，再应用 g (对应数学符号: g ∘ f)
example (f : M →* N) (g : N →* P) : M →* P := g.comp f
```

群环境下的性质、构造与策略

群同态本质上就是 `MonoidHom`。但得益于群的**可逆性**，我们在处理时拥有更多工具和简化的构造方式。

1. 自动化策略 (Tactics)

- `group`: 用于简化任意群中的恒等式。
- `abel`: 专用于交换加法群 (Abelian Groups)。

3. 简化的构造 (Simplified Construction)

虽然 `MonoidHom` 定义中只包含 `map_mul` 和 `map_one`，但在群中，保持乘法自动蕴含保持逆元。构造群同态时，我们不需要手动证明“保单位元”。

```
variable {G H : Type*} [Group G] [Group H]

-- 证明群恒等式
example (x y z : G) : x * (y * z) * (x * z)-1 * (x * y * x-1)-1 = 1 := by
  group

-- 自动保持逆元
example (x : G) (f : G →* H) : f (x-1) = (f x)-1 :=
  f.map_inv x

-- 仅通过保乘法构造群同态
def myGroupHom (f : G → H) (h : ∀ x y, f (x * y) = f x * f y) : G →* H :=
  MonoidHom.mk' f h
```

群同构 (Group Isomorphisms)

当两个群之间存在双射同态时，我们称其为同构。Lean 使用 `MulEquiv`（记作 $\simeq$ ）来表示。这种对象比同态更强，它包含反向映射，并且满足同类关系的三个基本性质：自反性、对称性和传递性。

```
variable {G H K : Type*} [Group G] [Group H] [Group K]

-- 自反性
#check MulEquiv.refl G      -- G ≈* G

-- 对称性: 如果 G ≈* H, 则 H ≈* G (即逆映射)
variable (f : G ≈* H)
#check f.symm                -- H ≈* G

-- 传递性: 如果 G ≈* H 且 H ≈* K, 则 G ≈* K
variable (g : H ≈* K)
#check f.trans g              -- G ≈* K

-- 同构与其逆映射组合后等于恒等映射
example : f.trans f.symm = MulEquiv.refl G := f.self_trans_symm
```

从双射构造同构:

```
-- 如果你有一个群同态 f, 而且它是双射, 那么 G 和 H 是同构的。
noncomputable def isoFromBij (f : G →* H) (h : Function.Bijective f) :
  G ≈* H := MulEquiv.ofBijective f h
```

注：Lean 是一个构造性证明助手，这意味着定义的函数通常需要是“可计算”的。虽然在数学上“双射必有逆映射”是显而易见的，但在逻辑上，从“存在一个逆映射”（命题）提取出“具体的逆函数”（数据）往往需要依赖选择公理。这一过程无法生成可执行的计算机代码，因此必须标记为 `noncomputable`。

11.2.2 子群 (Subgroups)

定义与基本性质

与群同态类似，Lean 中的子群也是一个**打包结构** (Bundled Structure)。一个 Subgroup G 不仅仅是一个集合，它还打包了封闭性公理 (包含单位元、乘法封闭、逆元封闭)。

```
variable {G : Type*} [Group G] (H : Subgroup G)

-- 1. 乘法封闭性
example {x y : G} (hx : x ∈ H) (hy : y ∈ H) : x * y ∈ H :=
  H.mul_mem hx hy

-- 2. 逆元封闭性
example {x : G} (hx : x ∈ H) : x⁻¹ ∈ H :=
  H.inv_mem hx
```

两个子群相等当且仅当它们包含相同的元素。我们可以使用 `ext` 将子群相等的证明转化为元素归属关系的双向推导。

```
-- 证明两个子群相等，等价于证明  $x \in H \leftrightarrow x \in K$ 
example (H K : Subgroup G) (h :  $\forall x, x \in H \leftrightarrow x \in K$ ) : H = K := by
  ext x
  exact h x
```

构造实例：有理数中的整数加法群

下面的例子展示了如何手动构造一个子群。我们将定义“由整数构成的有理数加法子群” (即在 $\mathbb{Q}$ 中的像)。这里涉及：

- **升格 (Coercion, $\uparrow$)**: 整数 $\mathbb{Z}$ 和有理数 $\mathbb{Q}$ 是不同的类型。我们需要使用 `Int.cast` 将整数“投射”到有理数中。
- **值域 (Range)**: `Set.range f` 表示函数 f 的像集。

```
example : AddSubgroup  $\mathbb{Q}$  where
  -- 1. 定义底层集合: 整数映射到有理数的值域
  carrier := Set.range (Int.cast :  $\mathbb{Z} \rightarrow \mathbb{Q}$ )

  -- 2. 加法封闭性
  -- 设  $a = \text{cast } n, b = \text{cast } m$ , 证明  $a + b$  也在值域中
  add_mem' := by
    rintro _ _ <n, rfl> <m, rfl>
    use n + m
    simp

  -- 3. 包含零元
  zero_mem' := by
    use 0
    simp

  -- 4. 取负封闭性
  neg_mem' := by
    rintro _ <n, rfl>
```

```
use -n
simp
```

子群作为类型

虽然子群 H 被定义为 G 的一项 (Term)，但 Lean 允许通过强制转换将其视为一个类型 (即子类型 $\{x : G // x \in H\}$)。由于子群本身也是群，Mathlib 已经注册了相关实例，我们可以直接使用 `inferInstance` 获取它。

```
-- 子群 H 继承了 G 的群结构
example (H : Subgroup G) : Group H := inferInstance
```

`inferInstance` 是一个指示 Lean 启动类型类推断 (Type Class Inference) 的函数。简单来说，它告诉 Lean: “请在已知的数据库中自动寻找并填入所需的实例”，而无需我们手动书写证明。

子群的格结构 (Lattice)

回忆 Day2 关于格的知识。Lean 中的所有子群构成了完全格，因此，我们可以直接应用任意关于格的通用引理 (例如交集的结合律、单调性等) 来处理子群。

- 偏序关系 ($\leq$): 对应集合的包含关系 ($\subseteq$)。
- 最大与最小: $\top$ (全群) 和 $\perp$ (平凡子群 $\{1\}$)。
- 交集 ($\cap$): 对应集合的交集。
- 生成子群 ($\sqcup$): 两个子群的并集通常不是子群。因此， $H \sqcup K$ 被定义为并集的闭包 (`Subgroup.closure`)。

```
-- 1. 偏序关系即集合包含. 在 Lean 中, 我们通常写  $H \leq K$  而非  $H \subseteq K$ 
example (H K : Subgroup G) : H ≤ K ↔ ∀ x, x ∈ H → x ∈ K :=
  SetLike.le_def -- 把“类似集合的结构”关系拆解为元素的包含关系。

-- 2. 交集的底层集合 = 集合的交集
example (H K : Subgroup G) :
  ((H ∩ K : Subgroup G) : Set G) = (H : Set G) ∩ (K : Set G) := rfl

-- 3. 并集生成的子群 = 集合并集的闭包
example {G : Type*} [Group G] (H K : Subgroup G) :
  ((H ∪ K : Subgroup G) : Set G) = Subgroup.closure ((H : Set G) ∪ (K : Set G)) := by
  rw [Subgroup.sup_eq_closure]

-- 4.  $\perp$  仅包含单位元
example (x : G) : x ∈ (⊥ : Subgroup G) ↔ x = 1 := Subgroup.mem_bot
```

同态对子群的作用: Map 与 Comap

我们可以利用群同态将子群在不同的群之间“推”和“拉”:

- **Map (推):** `Subgroup.map f H` 是 H 在 f 下的像 (前推)。
- **Comap (拉):** `Subgroup.comap f K` 是 K 在 f 下的原像 (拉回)。

特别地, 核 (Kernel) 和值域 (Range) 可以看作是 $\perp$ 和 $\top$ 的拉回与前推:

```
variable {N : Type*} [Group N] (f : G →* N)

-- Ker f = f-1({1})
example : MonoidHom.ker f = (⊥ : Subgroup N).comap f := rfl

-- Range f = f(G)
example : MonoidHom.range f = (⊤ : Subgroup G).map f := MonoidHom.range_eq_map f
```

11.2.3 正规子群与商群 (Quotient Groups)

在群论中, 商群 G/N 的构造依赖于子群 N 的正规性。在 Lean 中, 我们将其作为一个类型类 (Type Class) 实例来处理, 即只要我们在上下文中引入了 `[N.Normal]`, Mathlib 就会自动推断出商群 G/N 上应当具备的群结构。

商映射与泛性质

首先, 我们来看最基础的构造。商群在 Lean 中写作 G / N 。我们可以定义从原群 G 到商群的自然投影 (Canonical Projection), 通常记为 π 。更重要的是商群的泛性质 (Universal Property): 这是我们定义商群上同态的主要手段。如果有一个从 G 出发的同态 φ , 且它的核包含 N , 那么 φ 就可以唯一地“下降¹”为一个定义在商群 G/N 上的同态。

```
variable (N : Subgroup G) [N.Normal]

-- 1. 商映射  $\pi: G \rightarrow G/N$ 
-- QuotientGroup.mk' 构造了这一自然投影
def  $\pi : G \rightarrow* G / N := QuotientGroup.mk' N$ 

-- 2. 泛性质 (Lift)
-- 如果同态  $\varphi$  的核包含  $N$  (即  $N \leq \text{Ker } \varphi$ ), 则  $\varphi$  可以下降到商群
variable {M : Type*} [Group M] ( $\varphi : G \rightarrow* M$ )
example (h :  $N \leq \text{MonoidHom.ker } \varphi$ ) :  $G / N \rightarrow* M :=$ 
  QuotientGroup.lift N  $\varphi$  h
```

商群之间的映射 (Functoriality)

如果我们有二个商群 G/N 和 G'/N' , 以及一个原本在 G 和 G' 之间的同态 φ , 我们如何构造出商群之间的映射呢?

虽然数学上我们常说“ φ 将 N 映入 N' ”, 但在 Lean 中, Mathlib 推荐使用拉回 (Pullback) `comap` 的视角: 即 $N \leq \varphi^{-1}(N')$ 来避免 $\exists$ 。

```
variable {G' : Type*} [Group G'] (N' : Subgroup G') [N'.Normal]
variable ( $\varphi : G \rightarrow* G'$ )

-- 若  $\varphi$  将  $N$  映射入  $N'$  (即  $N \leq \varphi^{-1}(N')$ ), 则诱导商群间的映射
-- 如果我们写  $N.\text{map } \varphi \leq N'$ , 底层定义包含“存在量词”。而  $N \leq N'.\text{comap } \varphi$  是全称量词蕴含, 底层定义等价于  $\forall x, x \in N \rightarrow \varphi x \in N'$ 
```

¹这种将“定义在元素上的函数”转换为“定义在商类型上的函数”的操作, 在类型论中统称为提升 (Lift)

```
example (h : N ≤ N'.comap φ) : G / N →* G' / N' :=
  QuotientGroup.map N N' φ h
```

注：在数学上，如果两个正规子群 M 和 N 是同一个集合 ($M = N$)，我们会认为 G/M 和 G/N 是同一个群。但在 Lean 的类型论中，它们是不同的类型。因此，我们不能直接划等号，而是需要构造一个显式的同构。

```
variable (M N : Subgroup G) [M.Normal] [N.Normal]

-- 即使 M = N, G / M 与 G / N 也是不同的类型
example (h : M = N) : G / M ≃* G / N :=
  QuotientGroup.quotientMulEquivOfEq h
```

11.2.4 构造积群同态

最后补充一个实用的工具。在有限群论中，我们经常需要证明一个群同构于两个群的直积（例如 $G \cong H \times K$ ）。要构造这样的同构，第一步往往是构造一个从 G 射向 $H \times K$ 的同态。

Mathlib 提供了 `MonoidHom.prod` 函数，它允许我们将两个同态 $f : G \rightarrow H$ 和 $g : G \rightarrow K$ 组合成一个积同态。

```
variable {K : Type*} [Group K]
variable (f : G →* H) (g : G →* K)

-- 将两个同态组合成 G → H × K
-- 对应数学映射: x ↦ (f(x), g(x))
def prodHom' : G →* H × K := MonoidHom.prod f g

-- 典型应用: 利用商映射构造 G → (G / K) × (G / H)
-- 假设我们有正规子群 H, K, 这是证明 G ≅ H × K 的关键一步
variable (N1 N2 : Subgroup G) [N1.Normal] [N2.Normal]
def quotientProd : G →* (G / N1) × (G / N2) :=
  MonoidHom.prod (QuotientGroup.mk' N1) (QuotientGroup.mk' N2)

-- 进阶: 如果我们要证明同构, 可以使用 MulEquiv.prodCongr
#check MulEquiv.prodCongr -- 它表示: 如果 G ≅ H 且 G' ≅ K, 则 G × G' ≅ H × K
```

11.3 Exercises

```

section exercise1
variable (α : Type) (s t u : Set α)
/- 请尽量少使用 tactic 且不使用 `constrcutor` 完成以下四个题目. -/
example : s ∩ (s ∪ t) = s := sorry
example : s ∪ s ∩ t = s := sorry
example : s \ t ∪ t = s ∪ t := sorry
example (h : s ⊆ t) : s \ u ⊆ t \ u := sorry

/- 以下四个题目允许使用 tactic 或搜索定理来完成证明, 但仍请做到结构尽量精简. -/
example : s \ t ∪ t \ s = (s ∪ t) \ (s ∩ t) := by
  sorry
example : s ∩ t ∪ s ∩ u ⊆ s ∩ (t ∪ u) := by
  sorry
example : s \ (t ∪ u) ⊆ (s \ t) \ u := by
  sorry

variable (f : α → β) (v : Set β)
example : f '' s ⊆ v ↔ s ⊆ f ⁻¹' v := by
  sorry
end exercise1

section exercise2
example (a b c d x y : ℂ) (hx : x ≠ 0) (hy : y ≠ 0) :
  a + b / x + c / x ^ 2 + d / x ^ 3 = a + x⁻¹ * (y * b / y + (d / x + c) / x) := by
  sorry

example {x : ℝ} (h : x = 1 ∨ x = 2) : x ^ 2 - 3 * x + 2 = 0 := by
  sorry

example {n m : ℕ} : (n + m) ^ 3 = n ^ 3 + m ^ 3 + 3 * m ^ 2 * n + 3 * m * n ^ 2 := by
  sorry

example {a b c : ℝ} (ha : a ≤ b + c) (hb : b ≤ a + c) (hc : c ≤ a + b) :
  ∃ x y z, x ≥ 0 ∧ y ≥ 0 ∧ z ≥ 0 ∧ a = y + z ∧ b = x + z ∧ c = x + y := by
  set x := (b - a + c) / 2 with hx_def
  set y := (a - b + c) / 2 with hy_def
  set z := (a + b - c) / 2 with hz_def
  sorry

example (x : ℕ) (h : (x : ℚ) = 1) : x = 1 := by
  sorry

```

```

example {b : ℤ} : 0 ≤ max (-3) (b ^ 2) := by
  sorry

example (x : ℕ) : x ≥ 2 → x / 2 ≥ 1 := by
  sorry

end exercise₂

```

尝试定义由元素 x 诱导的共轭子群 xHx^{-1} 。

```

def conjugate (x : G) (H : Subgroup G) : Subgroup G where
  carrier := {a : G | ∃ h, h ∈ H ∧ a = x * h * x⁻¹}
  one_mem' := by
    dsimp
    use 1
    simp
  inv_mem' := by
    dsimp
    rintro _ ⟨h, h_in, rfl⟩
    use h⁻¹
    simp [h_in] -- 利用群的共轭性质
  mul_mem' := by
    dsimp
    rintro _ _ ⟨a, ha, rfl⟩ ⟨b, hb, rfl⟩
    use a * b
    simp [ha, hb] -- 利用 x(ab)x⁻¹ = (xax⁻¹)(xbx⁻¹)

```

The next two exercises derive a corollary of Lagrange' s lemma.

```

lemma eq_bot_iff_card {G : Type*} [Group G] {H : Subgroup G} :
  H = 1 ↔ Nat.card H = 1 := by
  suffices (∀ x ∈ H, x = 1) ↔ ∃ x ∈ H, ∀ a ∈ H, a = x by
    simpa [eq_bot_iff_forall, Nat.card_eq_one_iff_exists]
  sorry

#check card_dvd_of_le

lemma inf_bot_of_coprime {G : Type*} [Group G] (H K : Subgroup G)
  (h : (Nat.card H).Coprime (Nat.card K)) : H ∩ K = 1 := by
  sorry

```